

RESOURCE · WORKBOOK

조직 자동화 9선

기획·HR·마케팅 — 누구나 30분

9개 자동화 통합본 · 완성 코드 · 시트 템플릿 · 트러블슈팅

Google Apps Script · Gemini · Slack · 월 0원

강사 김창환 · 네다바웨이 · nedabah.org/auto

목차

1. **PLAN-01** · 회의록 → 액션아이템 자동 정리 (난이도 ★ · 30분)
2. **PLAN-02** · 경쟁사·뉴스 일일 다이제스트 (난이도 ★★ · 45분)
3. **PLAN-03** · 주간 KPI 자동 리포트 (난이도 ★★ · 60분)
4. **HR-01** · 신규 입사자 온보딩 키트 (난이도 ★★ · 60분)
5. **HR-02** · 휴가 신청 슬랙 승인 워크플로우 (난이도 ★★★ · 90분)
6. **HR-03** · 분기 펄스서베이 + AI 감성 분석 (난이도 ★★ · 45분)
7. **MKT-01** · 30일 콘텐츠 캘린더 자동 생성 (난이도 ★ · 30분)
8. **MKT-02** · 인입 리드 자동 스코어링·배정 (난이도 ★★ · 60분)
9. **MKT-03** · 리뷰·멘션 주간 다이제스트 (난이도 ★★ · 45분)

자료 허브: nedabah.org/auto · 강의 의뢰: nedabah.way@gmail.com

네다바웨이 · WORKBOOK

기획 자동화 (3선)


```

ui.ButtonSet.OK); const result = callGemini(text); appendToSheet(result, url); if (SLACK_HOOK)
postToSlack(result, url); ui.alert('완료', `${result.actions.length}개 액션아이템을 시트에 적재했습니다.`,
ui.ButtonSet.OK); } function callGemini(meetingText) { const prompt = ` 다음 회의록에서 정보를 추출하세
요. JSON만 반환하세요(설명 없이). 회의록: "${meetingText.slice(0, 30000)}" "" JSON 스키마:
{ "meetingName": "회의명", "meetingDate": "YYYY-MM-DD", "decisions": ["결정사항1", "결정사항2"],
"actions": [{"task": "액션내용", "owner": "담당자", "due": "YYYY-MM-DD 또는 빈문자열"}] } 규칙: - 담당자는 회
의록에 등장한 사람만 사용. 추정 금지. - 기한이 명시되지 않으면 빈문자열. - 결정사항과 액션아이템은 다르다(결정=합
의, 액션=실행). `; const url = `https://generativelanguage.googleapis.com/v1beta/models/

```

세션 30분 가이드

Step 1. 시트 만들기

```

{ method: 'post', contentType: 'application/json', payload: JSON.stringify({ contents: [{parts: [{text:
prompt}]}], generationConfig: {responseMimeType: 'application/json'}} }); const json =
JSON.parse(res.getContentText()); const content = json.candidates[0].content.parts[0].text; return
JSON.parse(content); } function appendToSheet(result, docUrl) { const sheet =
SpreadsheetApp.openById(SHEET_ID).getSheets()[0]; const decisionsStr = result.decisions.join(' |
3. URL에서 시트 ID 복사 (docUrl에서 docId를 추출)
Date().toISOString().slice(0,10), result.meetingName || '', decisionsStr, a.task, a.owner || '', a.due ||
'', '대기', docUrl || ''); } } function postToSlack(result, docUrl) { const lines = result.actions.map((a,i)
= `*${result.meetingName}* 회의 요약\n\n*결정사항*\n• ${result.decisions.join('\n• ')}\n\n*액션아이템*
1. https://aistudio.google.com/apikey 접속
2. Create API key 키 복사 (한번만 노출됨)
의록 Doc 오븐 메뉴 [자동화] → 액션아이템 추출] 클릭 15초 대기 → Sheet에 5~7행 자동 추가되는 모습 시연
Slack 채널에 자동 발송된 카드 시연 잘못된 행을 직접 수정하며 "AI는 보조, 사람은 검토" 메시지 강조 트러블슈팅 증상
원인 해결
Step 3. Slack Webhook (선택)
조 + responseMimeType 확인 한국어 담당자 이름 깨짐 UTF-8 처리 실패 Apps Script v8 런타임 사용 (기본값) 시
트에 반 행만 추가 actions 배열 0개 회의록이 짝거나 액션이 없는 경우 정상 동작 Gemini 429 에러 무료 티어 분당 한
도 초과 Utilities.sleep(4500) 호출 사이 추가 응용 아이디어 (강의 후반 토론용) 임원 회의 보안 강화: Sheet ID를 권
한 그룹만 접근 가능으로 설정해 채널별 다국어 회의 Webhook URL 변환 회의 원문 언어로" 추가 CRM 연동: 액션의
owner가 영업일 경우 Salesforce/HubSpot Task로 자동 생성 음성 회의록: Google Meet 녹화 → Whisper API 전
사 → 본 스크립트로 연결

```

Step 2. Gemini API 키 발급

1. <https://aistudio.google.com/apikey> 접속

2. Create API key 키 복사 (한번만 노출됨)

의록 Doc 오븐 메뉴 [자동화] → 액션아이템 추출] 클릭 15초 대기 → Sheet에 5~7행 자동 추가되는 모습 시연

Slack 채널에 자동 발송된 카드 시연 잘못된 행을 직접 수정하며 "AI는 보조, 사람은 검토" 메시지 강조 트러블슈팅 증상

원인 해결

Step 3. Slack Webhook (선택)

조 + responseMimeType 확인 한국어 담당자 이름 깨짐 UTF-8 처리 실패 Apps Script v8 런타임 사용 (기본값) 시

트에 반 행만 추가 actions 배열 0개 회의록이 짝거나 액션이 없는 경우 정상 동작 Gemini 429 에러 무료 티어 분당 한

도 초과 Utilities.sleep(4500) 호출 사이 추가 응용 아이디어 (강의 후반 토론용) 임원 회의 보안 강화: Sheet ID를 권

한 그룹만 접근 가능으로 설정해 채널별 다국어 회의 Webhook URL 변환 회의 원문 언어로" 추가 CRM 연동: 액션의

owner가 영업일 경우 Salesforce/HubSpot Task로 자동 생성 음성 회의록: Google Meet 녹화 → Whisper API 전

사 → 본 스크립트로 연결

Step 4. Apps Script 붙여넣기

1. Google Sheet에서 확장 프로그램 → Apps Script
2. `script.gs` 내용 붙여넣기 (아래)
3. 프로젝트 설정 → 스크립트 속성에 다음 추가: - `GEMINI_API_KEY` = (발급받은 키) - `SHEET_ID` = (시트 ID) - `SLACK_WEBHOOK` = (선택)
4. 트리거 추가: `onMeetingDocOpen` → 시간 기반 → 매일 오전 9시 또는 **Docs 메뉴에서 직접 실행**(아래 `installDocMenu` 사용)

Step 5. 회의록 작성 규칙 (사용자 약속)

- 문서 첫 줄: 회의명: 주간 마케팅 회의 / 2026-05-04
- 본문은 자연어 그대로 작성 (별도 양식 불필요)
- 작성 완료 후 메뉴  자동화 → 액션아이템 추출 클릭

ui.ButtonSet.OK); const result = callGemini(text); appendToSheet(result, url); if (SLACK_HOOK)
 postToSlack(result, url); ui.alert('완료', `\${result.actions.length}개 액션아이템을 시트에 적재했습니다.` ,
 ui.ButtonSet.OK); } function callGemini(meetingText) { const prompt = ` 다음 회의록에서 정보를 추출하세
 요. JSON만 반환하세요(설명 없이). 회의록: "\${meetingText.slice(0, 30000)}" JSON 스키마:
 { "meetingName": "회의명", "meetingDate": "YYYY-MM-DD", "decisions": ["결정사항1", "결정사항2"],
 "actions": [{ "task": "액션내용", "owner": "담당자", "due": "YYYY-MM-DD 또는 빈문자열" }] } 규칙: - 담당자는 회
 의록에 등장한 사람만 사용. 부정 금지. - 기한이 명시되지 않으면 빈문자열. - 결정사항과 액션아이템은 다르다(결정=합
 의, 액션=실행). `; const url = `https://generativelanguage.googleapis.com/v1beta/models/
 gemini-2.5-flash:generateContent?key=\${GEMINI_KEY}`; const res = UrlFetchApp.fetch(url,
 { method: 'post', contentType: 'application/json', payload: JSON.stringify({ contents: [{parts: [{text:
 prompt}]}], generationConfig: {responseMimeType: 'application/json'}} }); const json =
 JSON.parse(res.getContentText()); const content = json.candidates[0].content.parts[0].text; return
 JSON.parse(content); } function appendToSheet(result, docUrl) { const sheet =
 SpreadsheetApp.openById(SHEET_ID).getSheets()[0]; const decisionsStr = result.decisions.join(' |
 '); result.actions.forEach(a => { sheet.appendRow([result.meetingDate || new
 Date().toISOString().slice(0,10), result.meetingName || '', decisionsStr, a.task, a.owner || '', a.due ||
 '', '대기', docUrl]); }); } function postToSlack(result, docUrl) { const lines = result.actions.map((a,i)
 => `\${i+1}. \${a.task} - \${a.owner || '미지정'} \${a.due ? ` (~\${a.due})` : ''}`).join('\n'); const text = `
 📝 *\${result.meetingName}* 회의 요약\n\n*결정사항*\n• \${result.decisions.join('\n• ')}\n\n*액션아이템*
 \n\${lines}\n\n🔗 \${docUrl}`; UrlFetchApp.fetch(SLACK_HOOK, { method: 'post', contentType:
 'application/json', payload: JSON.stringify({text}) }); } 강의 시연 시나리오 (10분) 강사가 미리 준비한 가짜 회
 의록 Doc 오픈 메뉴 [🔗 자동화 → 액션아이템 추출] 클릭 15초 대기 → Sheet에 5~7행 자동 추가되는 모습 시연
 Slack 채널에 자동 발송된 카드 시연 잘못된 행을 직접 수정하며 "AI는 보조, 사람은 검토" 메시지 강조 트러블슈팅 증상
 원인 해결 Cannot read properties of undefined Gemini 응답이 JSON이 아님 프롬프트 끝에 "JSON만 반환" 강
 조 + responseType 확인 한국어 담당자 이름 깨짐 UTF-8 처리 실패 Apps Script v8 런타임 사용 (기본값) 시
 트에 빈 행만 추가 actions 배열 0개 회의록이 짧거나 액션이 없는 경우, 정상 동작 Gemini 429 에러 무료 티어 분당 한
 도 초과 Utilities.sleep(4500) 호출 사이 추가 응용 아이디어 (강의 후반 토론용) 임원 회의 보안 강화: Sheet ID를 권
 한 그룹으로 잠그고 Slack은 비공개 채널만 다국어 회의: 프롬프트에 "답변은 회의 원문 언어로" 추가 CRM 연동: 액션의
 owner가 영업일 경우 Salesforce/HubSpot Task로 자동 생성 음성 회의록: Google Meet 녹화 → Whisper API 전
 사 → 본 스크립트로 연결


```

ui.ButtonSet.OK); const result = callGemini(text); appendToSheet(result, url); if (SLACK_HOOK)
postToSlack(result, url); ui.alert('완료', ` ${result.actions.length}개 액션아이템을 시트에 적재했습니다.` ,
ui.ButtonSet.OK); } function callGemini(meetingText) { const prompt = ` 다음 회의록에서 정보를 추출하세
요. JSON만 반환하세요(설명 없이). 회의록: "" ${meetingText.slice(0, 30000)} "" JSON 스키마:
{ "meetingName": "회의명", "meetingDate": "YYYY-MM-DD", "decisions": ["결정사항1", "결정사항2"],
"actions": [{"task": "액션내용", "owner": "담당자", "due": "YYYY-MM-DD 또는 빈문자열" } ] } 규칙: - 담당자는 회
의록에 등장한 사람만 사용. 추정 금지. - 기한이 명시되지 않으면 빈문자열. - 결정사항과 액션아이템은 다르다(결정=합
의, 액션=실행). `; const url = `https://generativelanguage.googleapis.com/v1beta/models/
gemini-2.5-flash:generateContent?key=${GEMINI_KEY}`; const res = UrlFetchApp.fetch(url,
{ method: 'post', contentType: 'application/json', payload: JSON.stringify({ contents: [{parts: [{text:
prompt}]}], generationConfig: {responseMimeType: 'application/json'} }) }); const json =
JSON.parse(res.getContentText()); const content = json.candidates[0].content.parts[0].text; return
JSON.parse(content); } function appendToSheet(result, docUrl) { const sheet =
SpreadsheetApp.openById(SHEET_ID).getSheets()[0]; const decisionsStr = result.decisions.join(' |
'); result.actions.forEach(a => { sheet.appendRow([ result.meetingDate || new
Date().toISOString().slice(0,10), result.meetingName || '', decisionsStr, a.task, a.owner || '', a.due ||
'', '대기', docUrl ]); }); } function postToSlack(result, docUrl) { const lines = result.actions.map((a,i)
=> `${i+1}. ${a.task} - ${a.owner || '미지정'} ${a.due ? ` (${a.due})` : ''}`).join("\n"); const text = `
* ${result.meetingName} * 회의 요약\n\n* 결정사항*\n\n* ${result.decisions.join('\n• ')}\n\n* 액션아이템
*\n\n${lines}\n\n🔗 ${docUrl}`; UrlFetchApp.fetch(SLACK_HOOK, { method: 'post', contentType:
'application/json', payload: JSON.stringify({text}) }); } 강의 시연 시나리오 (10분) 강사가 미리 준비한 가짜 회
의록 Doc로든 메뉴 [🔍 자동화 → 액션아이템 추출] 클릭 15초 대기 → Sheet에 3~7행 자동 추가 되는 모습 시연
Slack 채널에 자동 발송된 카드 시연 잘못된 행을 직접 수정하며 "AI는 보조, 사람은 검토" 메시지 강조
트러블슈팅 증상 원인 해결 Cannot read properties of undefined Gemini 응답이 JSON이 아님 프롬프트 끝에 "JSON만 반환" 강
조 + responseMimeType 확인 한국어 담당자 이름 깨짐 UTF-8 처리 실패 Apps Script v8 런타임 사용 (기본값) 시
트에 빈 행만 추가 actions 배열 0개 회의록이 짧거나 액션이 없는 경우, 정상 동작 Gemini 429 에러 무료 티어 분당 한
도 초과 시 Utilities.sleep(4500) 후로 시연 추가 응용 아이디어 (강의 후반 토론용) 임원 회의 보안 강화: Sheet ID를 권
한 그룹으로 잠그고 Slack은 비공개 채널만 다국어 회의: 프롬프트에 "답변은 회의 원문 언어로" 추가 CRM 연동: 액션의
owner가 영업일 경우 Salesforce/HubSpot Task로 자동 생성 음성 회의록: Google Meet 녹화 → Whisper API 전
사 → 본 스크립트로 연결

```

• **다국어 회의:** 프롬프트에 "답변은 회의 원문 언어로" 추가

• **CRM 연동:** 액션의 owner가 영업일 경우 Salesforce/HubSpot Task로 자동 생성

• **음성 회의록:** Google Meet 녹화 → Whisper API 전사 → 본 스크립트로 연결


```

    {muteHttpExceptions: true}).getContentText()); const doc = XmlService.parse(xml); const items =
    doc.getRootElement().getChild('channel').getChildren('item'); return items.map(it => ({ title:
    it.getChildText('title') || '', url: it.getChildText('link') || '', pub: new Date(it.getChildText('pubDate')
    || Date.now()), desc: (it.getChildText('description') || '').replace(/<[^>]+>/g, '').slice(0,
    500) })).filter(it => it.pub >= since); } catch(e) { Logger.log(`RSS 실패 ${url}: ${e}`); return []; } }
function deduplicate(items) { const seen = new Set(); return items.filter(it => { const key =
    it.title.replace(/<\/a>|'|"/g, '') + it.url; if (seen.has(key)) return false; seen.add(key); return true; }); }
function summarizeBatch(items) { const prompt = ` 다음 뉴스 ${items.length}건을 각각 한국어 1~2문장으
    로 요약하세요. - 핵심 사실만, 추측 금지 - "~다"체로 통일 - JSON 배열만 반환: [{"summary":"..."},...] 순서 유지.
    뉴스: ${items.map((it,i)=>`[${i+1}] ${it.title}\n${it.desc}`).join('\n\n')}`; const url = `https://
    generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${
    GEMINI_KEY}`; const res = UrlFetchApp.fetch(url, { method: 'post', contentType: 'application/json',
    payload: JSON.stringify({ contents:[{parts:[{text: prompt}]}], generationConfig:{responseMimeType:
    'application/json'}})); const arr =
    JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); return
    items.map((it, i) => ({...it, summary: (arr[i]||{}).summary || ''})); } function buildDigestHTML(items)
    { const grouped = {}; items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
    grouped[it.category].push(it); }); let html = `<div style="font-family:'Noto Sans KR',sans-serif;max-
    width:680px;line-height:1.6;"> <h2 style="border-bottom:2px solid #b45309;padding-bottom:.
    4rem;">📰 데일리 다이제스트</h2> <p style="color:#6a604f;">${new Date().toLocaleDateString('ko-
    KR')} · 총 ${items.length}건</p>`; for (const cat in grouped) { html += `<h3 style="margin-top:
    1.5rem;color:#3a322a;">${cat}</h3><ul>`; grouped[cat].forEach(it => { html += `<li style="margin-
    bottom:.6rem;"><a href="${it.url}" style="color:#b45309;text-decoration:none;font-weight:600;">${
    it.title}</a><br><span style="color:#555;font-size:.95em;">${it.summary}</span></li>`; }); html +=
    `</ul>`; } html += `<p style="color:#999;font-size:.85em;margin-top:2rem;">자동 생성 · 출처는 각 링크
    참조</p></div>`; return html; } function htmlToSlackText(items, date) { const grouped = {};
    items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
    grouped[it.category].push(it); }); let txt = `*📰 ${date} 데일리 다이제스트* (${items.length}건)\n`; for
    (const cat in grouped) { txt += `\n*${cat}*\n`; grouped[cat].forEach(it => txt += `• <${it.url}>|
    ${it.title}> — ${it.summary}\n`); } return txt; } 강의 시연 포인트 keywords 시트의 한 행을 추가/수정하며 "코드
    한 줄도 안 고치고 모니터링 대상 변경" 시연 dailyDigest() 수동 실행 → 1분 내 메일/Slack에 도착하는 모습 보여주기
    키워드 잘못 잡혔을 때 → AI 요약 한 줄로 빠르게 무관함 판별 가능 강조 트러블슈팅 증상 원인 해결 RSS 0건 키워드 너
    무 줍음 / RSS URL 만료 Google News RSS는 한국어 검색에 &hl=ko&gl=KR&ceid=KR:ko 추가 Gemini 응답 잘림
    items가 50건 초과 MAX_PER_KEYWORD 줄이거나 summarizeBatch를 청크로 분할 Gmail 송신 실패 일일 한도(개
    인계정 100건) Workspace 계정 사용 또는 Slack만 사용 트리거 시간 어긋남 Apps Script는 ±1시간 업무 시작 1시간
    전으로 설정 응용 아이디어 카테고리별 가중치: keywords 시트에 priority 컬럼 추가 → 높은 우선순위는 별도 강조
    PDF 보고서: HTML을 DocumentApp으로 변환해 주간 PDF로 자동 보관 Slack 인터랙션: 다이제스트 카드에 "관심 /
    무관심" 버튼 추가하여 학습 데이터로 활용 자사 멘션 알림: 자사 키워드 카테고리만 즉시 알림(트리거 분리)으로 위기 대
    응 단축

```

```

{muteHttpExceptions: true}).getContentText(); const doc = XmlService.parse(xml); const items =
doc.getRootElement().getChild('channel').getChildren('item'); return items.map(it => ({ title:
it.getChildText('title') || '', url: it.getChildText('link') || '', pub: new Date(it.getChildText('pubDate')
|| Date.now()), desc: (it.getChildText('description') || '').replace(/<[^>+>/g, '').slice(0,
500) })).filter(it => it.pub >= since); } catch(e) { Logger.log(`RSS 실패 ${url}: ${e}`); return []; } }
function deduplicate(items) { const seen = new Set(); return items.filter(it => { const key =
it.title.replace(/<[^>+>/g, ''); if (seen.has(key)) return false; seen.add(key); return true; }); }
const PROPS = PropertiesService.getScriptProperties();
function GEMINI_KEY = PROPS.getProperty('GEMINI_API_KEY');
const SHEET_ID = PROPS.getProperty('SHEET_ID');
const EMAIL_TO = PROPS.getProperty('RECIPIENT_EMAIL');
const SLACK_HOOK = PROPS.getProperty('SLACK_WEBHOOK');
const LOOKBACK_HOURS = 24;
const MAX_PER_KEYWORD = 8;
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); return
items.map(it => ({ summary: (arr[i] || []).summary || '' })); } function buildDigestHTML(items)
{ const grouped = {}; items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
grouped[it.category].push(it); }); let html = `<div style="font-family: 'Noto Sans KR', sans-serif; max-
width: 680px; line-height: 1.6;"><h2 style="border-bottom: 2px solid #b45309; padding-bottom:
4rem;">데일리 다이제스트</h2> <p style="color: #6a604f;">${new Date().toLocaleDateString('ko-
KR')} <span style="color: #6a604f;">${(items.length > 0 ? '총 ' + items.length + '건' : '없음')}</span></p>`;
const since = new Date(Date.now() - (LOOKBACK_HOURS * 3600 * 1000));
const collected = [];
for (let i = 1; i <= MAX_PER_KEYWORD; i++) {
const [category, keyword, rssurl] = kw[i];
if (!rssurl) continue;
const items = fetchRSS(rssurl, since).slice(0, MAX_PER_KEYWORD);
items.forEach(it => collected.push({ ...it, category, keyword }));
Utilities.sleep(800);
}
const txt = `<div> ${date} 데일리 다이제스트 </div>`;
const html = buildDigestHTML(collected);
return txt; }
const keywords = keywordsSheet.getValues();
const today = new Date().toISOString().slice(0, 10);
const summarized = [];
for (let i = 0; i < dedup.length; i += 50) {
const chunk = dedup.slice(i, i + 50);
summarized.push(...summarizeBatch(chunk));
if (i + 50 < dedup.length) Utilities.sleep(2000);
}
const html = buildDigestHTML(summarized);
// 로그 적재
const today = new Date().toISOString().slice(0, 10);
summarized.forEach(s => log.appendRow([today, s.title, s.url, s.category, s.summary]));
if (EMAIL_TO) {
GmailApp.sendEmail(EMAIL_TO, `데일리리 ${today} 경쟁사·산업 다이제스트`, '', {htmlBody:
html});
}
if (SLACK_HOOK) {
UrlFetchApp.fetch(SLACK_HOOK, {
method: 'post', contentType: 'application/json',
payload: JSON.stringify({text: htmlToSlackText(summarized, today)})
});
}
}
function fetchRSS(url, since) {

```



```

    {muteHttpExceptions: true}).getContentText(); const doc = xmlService.parse(xml); const items =
    doc.getRootElement().getChild('channel').getChildren('item'); return items.map(it => ({ title:
    it.getChildText('title') || '', url: it.getChildText('link') || '', pub: new Date(it.getChildText('pubDate')
    || Date.now()), desc: (it.getChildText('description') || '').replace(/<[>]+>/g, '').slice(0,
    500) })).filter(it => it.pub >= since); } catch(e) { Logger.log(`RSS 실패 ${url}: ${e}`); return []; } }

function deduplicate(items) { const seen = new Set(); return items.filter(it => { const key =
it.title.replace(/s+/g, '').slice(0, 30); if (seen.has(key)) return false; seen.add(key); return true; }); }
function summarizeBatch(items) { const prompt = `다음 뉴스 ${items.length}건을 각각 한국어 1~2문장으로
로 요약하세요. - 핵심 사실만, 출처 금지 - ~단 체로 통일 - JSON 배열만 반환: [{"summary": "..."},...] 순서 유지.
뉴스: ${items.map((it, i) => `["${i+1}"] ${it.title}\n${it.desc}`).join('\n\n')}`; const url = `https://
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${
GEMINI_KEY}`; const res = UrlFetchApp.fetch(url, { method: 'post', contentType: 'application/json',
payload: JSON.stringify({contents:[{parts:[{text: prompt}]}], generationConfig:{responseMimeType:
'application/json'}})); const arr =
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); return
items.map((it, i) => ({...it, summary: (arr[i]||{}).summary || ''})); } function buildDigestHTML(items)
{ const grouped = {}; items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
Logger.log(`RSS 실패 ${url}: ${e}`);
grouped[it.category].push(it); }); let html = `<div style="font-family:'Noto Sans KR',sans-serif;max-
width:680px;line-height:1.6;"><h2 style="border-bottom:2px solid #b45309;padding-bottom:
4rem;">📰 데일리 다이제스트</h2> <p style="color:#6a604f;">${new Date().toLocaleDateString('ko-
KR')} · 총 ${items.length}건</p>`; for (const cat in grouped) { html += `<h3 style="margin-top:
1.5rem;color:#3a322a;">${cat}</h3><ul>`; grouped[cat].forEach(it => { html += `<li style="margin-
bottom:.6rem;"><a href="${it.url}" style="color:#b45309;text-decoration:none;font-weight:600;">${
it.title}</a><br><span style="color:#555;font-size:.95em;">${it.summary}</span></li>`; }); html +=
`</ul>`; html += `<p style="color:#999;font-size:.85em;margin-top:2rem;">자동 생성 · 출처는 각 링크
점조</p></div>`; return html; } function htmlToSlackText(items, date) { const grouped = {};
items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
grouped[it.category].push(it); }); let txt = `*📰 ${date} 데일리 다이제스트* (${items.length}건)\n`; for
(const cat in grouped) { txt += `\n*${cat}*\n`; grouped[cat].forEach(it => txt += `· <${it.url}>${
it.title}> - ${it.summary}\n`); } return txt; } 강의 시연 포인트 keywords 시트의 한 행을 추가/수정하며 "코드
한 줄도 안 고치고 모니터링 대상 변경" 시연 dailyDigest() 수동 실행 → 1분 내 메일/Slack에 도착하는 모습 보여주기
키워드 잘못 잡혔을 때 → AI 요약 한 줄로 빠르게 무관함 판별 가능 강조 트러블슈팅 증상 원인 해결 RSS 0건 키워드 너
무 좁음 → RSS 키워드 Google News RSS는 한국어 검색에 &hl=ko&gl=KR&ceid=KR:ko 추가 Gemini 응답 잘림
items가 50건 초과 MAX_PER_KEYWORD 줄이거나 summarizeBatch를 청크로 분할 Gmail 송신 실패 일일 한도(개
인 계정 0건) → Workspace 계정 사용 또는 Slack만 사용 트러거 시연 미국남 Apps Script는 ±1시간 업무 시작 1시간
전으로 실행 자동 하거나 카테고리별 가중치: keywords 시트에 priority 컬럼 추가 → 높은 우선순위는 별도 강조
PDF 보관 시 서버 VM을 DocumentApp으로 변환해 주간 PDF로 자동 보관 Slack 인터랙션: 다이제스트 카드에 "관심 /
무관함" 버튼 추가 하여 학습 데이터로 활용 자사 랜선 알림: 자사 키워드 카테고리만 즉시 알림(트리거 분리)으로 위기 태
순서 유지.
등 단축

뉴스:
${items.map((it,i)=>`[${i+1}] ${it.title}\n${it.desc}`).join('\n\n')}
`;

const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
flash:generateContent?key=${GEMINI_KEY}`;
const res = UrlFetchApp.fetch(url, {
method: 'post', contentType: 'application/json',
payload: JSON.stringify({
contents:[{parts:[{text: prompt}]}],
generationConfig:{responseMimeType: 'application/json'}
})
});
const arr =
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text);
return items.map((it, i) => ({...it, summary: (arr[i]||{}).summary || ''}));
}

function buildDigestHTML(items) {
const grouped = {};
items.forEach(it => {
grouped[it.category] = grouped[it.category] || [];
grouped[it.category].push(it);
});
}

```

```

    {muteHttpExceptions: true}).getContentText(); const doc = XmlService.parse(xml); const items =
    doc.getRootElement().getChild('channel').getChildren('item'); return items.map(it => ({ title:
    it.getChildText('title') || '', url: it.getChildText('link') || '', pub: new Date(it.getChildText('pubDate')
    || Date.now()), desc: (it.getChildText('description') || '').replace(/<[^>+>/g, '').slice(0,
    500) })).filter(it => it.pub >= since); } catch(e) { Logger.log(`RSS 실패 ${url}: ${e}`); return []; } }

function deduplicate(items) { const seen = new Set(); return items.filter(it => { const key =
it.title.replace(/s/g, '').slice(0, 30); if (seen.has(key)) return false; seen.add(key); return true; }); }

function summarizeBatch(items) { const prompt = `다음 뉴스 ${items.length}건을 각각 한국어 1~2문장으
로 요약하세요. - 핵심 사실만, 주석 금지 - ~다 제로 통일 - JSON 배열만 반환: [{ "summary": "...", ... } 순서 유지.
h2>
뉴스: ${items.map((it,i) => `["${i+1}" "${it.title}\n${it.desc}").join('\n\n')}`; const url = `https://
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${
{GEMINI_KEY}}`; const res = UrlFetchApp.fetch(url, { method: 'post', contentType: 'application/json',
payload: JSON.stringify({ contents: [{ parts: [{ text: prompt }] }], generationConfig: { responseMimeType:
'application/json' } })); const arr =
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); return
items.map((it,i) => ({ it.summary: arr[i][0].summary || '' })); } function buildDigestHTML(items)
{ const grouped = {}; items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
style="color:#555;font-size:.95em;">${it.summary}</span></li>`; }); html +=
`</ul>`; } html += `<p style="color:#999;font-size:.85em;margin-top:2rem;">자동 생성 · 출처는 각 링크
참조</p></div>`; return html; } function htmlToSlackText(items, date) { const grouped = {};
items.forEach(it => { grouped[it.category] = grouped[it.category] || [];
grouped[it.category].push(it); }); let txt = `*📰 ${date} 데일리 다이제스트* (${items.length}건)\n`; for
const cat in grouped { txt += `<div> ${cat}</div>`; grouped[cat].forEach(it => { txt += `<div>
grouped[cat].forEach(it => txt += `<div>
txt += `<div>
grouped[cat].forEach(it => txt += `<div>
return txt;
}

PDF 보고서: HTML을 DocumentApp으로 변환해 주간 PDF로 자동 보관 Slack 인터랙션: 다이제스트 카드에 "관심 /
무관심" 버튼 추가하여 학습 데이터로 활용 자사 멘션 알림: 자사 키워드 카테고리한 즉시 알림(드러거 분리)으로 위키 내
응 단축

```

강의 시연 포인트

1. keywords 시트의 한 행을 추가/수정하며 "코드 한 줄도 안 고치고 모니터링 대상 변경" 시연
2. dailyDigest() 수동 실행 → 1분 내 메일/Slack에 도착하는 모습 보여주기
3. 키워드 잘못 잡혔을 때 → AI 요약 한 줄로 빠르게 무관함 판별 가능 강조


```
meta.target; return reached ? '✅ 달성' : '⚠️ 미달'; } function aiComment(summary) { const prompt = `당신은 경영진 보고를 작성하는 시니어 분석가입니다. 다음 주간 KPI를 보고 한국어로 간결하게 작성하세요. 규칙: - 사실만, 추측 금지 - 5문장 이내 - 1) 가장 좋은 변화 1건 2) 가장 우려되는 변화 1건 3) 다음 주 권고 1건 - "~합니다"체 데이터: ${JSON.stringify(summary, null, 2)} JSON으로 반환: {"highlight":"...", "concern":"...", "recommendation":"..."} `; const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GOEMINI_KEY}&context=${JSON.stringify(summary, null, 2)}&method=post`, contentType:'application/json', payload: JSON.stringify({ contents:[{text:prompt}]}), generationConfig: {responseMimeType:'application/json'} })); return
```

PLAN-03 주간 KPI 자동 리포트

시각 변화: 이상, 다음 주 권고를 함께 작성한 한 장 KPI 리포트를 매주 임원에게 발송합니다.

```
buildReportHTML(summary, insight, weekStart) { let rows = summary.map(s => { const arrow = s.wow > 1 ? '▲' : s.wow < -1 ? '▼' : '—'; const wowStr = s.wow ? ` ${s.wow.toFixed(1)}%` : '—'; return `<tr> <td style="padding:.5rem;border-bottom:1px solid #eee;">${s.indicator}</td> <td style="padding:.5rem;border-bottom:1px solid #eee;text-align:right;">${s.thisAvg.toFixed(1)} $</td> <td style="padding:.5rem;">60분</td> <td style="padding:.5rem;">KPI 적재된 Google</td> <td style="padding:.5rem;">0원</td> <td style="padding:.5rem;">Sheet + Gemini API</td> <td style="padding:.5rem;">지표별 표</td> <td style="padding:.5rem;">지표</td> <td style="padding:.5rem;">이변주 평균</td> <td style="padding:.5rem;">WoW</td> <td style="padding:.5rem;">목표</td> <td style="padding:.5rem;">상태</td></tr>`</td> <td style="padding:.5rem;">이탈을 같은 부서별 지표로 같은 코드로 처리됨</td> <td style="padding:.5rem;">시연</td> <td style="padding:.5rem;">최소 14일치 데이터 누적 후 사용 트리거가 한 번도 안 돌아감 인증 미완료 Apps Script에서 한 번 수동 실행하여 권한 부여 임부지표, 능력지표, 역량지표 등록 안 됨 kpi_meta에 추가하지 않아도 표시되지만, 목표/상태가 비어 보임 응용 아이디어 부서별 분기: 시트에 부서 컬럼 추가 → 부서별 메일을 다른 수신자에 발송 Looker Studio 연동: Apps Script가 Sheet에 적재한 결과를 Looker가 시각화 이상 감지 강화: 이번주 값이 표준편차 ±2σ 벗어나면 즉시 임원 알림 분리 OKR 연결: kpi_meta에 okr_id 컬럼 추가하여 OKR 트래킹과 통합
```

한 줄 핵심: 흠어진 KPI 시트를 매주 월요일 새벽에 모아, 시각 변화 포인트와 다음 주 권고를 작성한 한 장 리포트로 임원에게 발송한다.

왜 이 자동화인가

수동(Before) 자동(After)

주간 리포트 작성 3~5시간/주 0분

데이터 출처 일관성 사람마다 다른 단일 수수

인사이트 깊이 시간 압박으로 얕음 시각 변동 이상 자동 검출

1년 시간 절감 약 200시간/팀

적용 시나리오: 경영지원 전략 기획, 사업부장 보고, 이사회 사전자료

구성요소

• Google Sheet (KPI 원천 — 일별 적재 가정)

• Google Slides 또는 Gmail (리포트 출력)

• Apps Script + Gemini API

• Slack (선택)

셋업 가이드

Step 1. KPI 시트 형식 (예시)

탭 kpi_daily : | 날짜 | 지표 | 값 | 단위 | 주관 | |---|---|---|---|---| | 2026-04-28 | 신규가입 | 412 | 명 | 마케팅

| | 2026-04-28 | DAU | 18900 | 명 | 프로덕트 | | 2026-04-28 | 결제전환율 | 3.4 | % | 영업 |

```

meta.target; return reached ? '✅ 달성' : '⚠️ 미달'; } function aiCommentary(summary) { const prompt
= `당신은 경영진 보고를 작성하는 시니어 분석가입니다. 다음 주간 KPI를 보고 한국어로 간결하게 작성하세요. 규칙: -
사실만, 추측 금지 - 5문장 이내 - 1) 가장 좋은 변화 1건 2) 가장 우려되는 변화 1건 3) 다음 주 권고 1건 - "~합니다"체
데이터: ${JSON.stringify(summary, null, 2)} JSON으로 반환:
{"highlight":"...", "concern":"...", "recommendation":"..."} `; const url = `https://
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${
[속한행] 한 지표의 하루 값 시트는 "단일 truth source"
{GEMINI_KEY}&contentType=application/json&method=post', contentType:'application/json',
payload: JSON.stringify({ contents:[{parts:[{text:prompt}]}], generationConfig:
{responseMimeType:'application/json'}}) }); return
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
buildReportHTML(summary, insight, weekStart) { let rows = summary.map(s => { const arrow =
s.wow > 1 ? '▲' : s.wow < -1 ? '▼' : '—'; const wowStr = s.wow ? `${s.wow.toFixed(1)}%` : '—';
return `<tr><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: left;">지표</td><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: right;">이전</td><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: right;">이번주 평균</td><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: right;">지난주 평균</td><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: right;">WoW</td><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: right;">목표</td><td style="padding: 5rem; border-bottom: 1px solid #eee; text-align: left;">상태</td></tr>`
});.join(""); return `<div style="font-family:'Noto Sans KR', sans-serif; max-width: 760px; line-height: 1.6; color: #222;"><h2 style="border-bottom: 2px solid #b45309; padding-bottom: .5rem;">주간 KPI 리포트</h2><p style="color: #6a604f;">기준 주: ${fmt(weekStart)} ~ ${fmt(new Date())}</p><h3>🔴 핵심 코멘트</h3><p><b>가장 좋은 변화 —</b>${insight.highlight}</p><p><b>우려 포인트 —</b>${insight.concern}</p><p><b>다음 주 권고 —</b>${insight.recommendation}</p><h3 style="margin-top: 1.5rem;">📊 지표별 표</h3><table style="width: 100%; border-collapse: collapse; font-size: .95em;"><thead style="background: #fbf6ec;"><tr><th style="padding: .5rem; text-align: left;">지표</th><th style="padding: .5rem; text-align: right;">이번주 평균</th><th style="padding: .5rem; text-align: right;">지난주 평균</th><th style="padding: .5rem; text-align: right;">WoW</th><th style="padding: .5rem; text-align: right;">목표</th><th style="padding: .5rem; text-align: left;">상태</th></tr></thead><tbody>${rows}</tbody></table><p style="color: #999; font-size: .85em; margin-top: 2rem;">자동 생성 · 원천 데이터: 회사 KPI 시트</p></div>`; } function buildSlack(summary, insight) { const top =
summary.map(s => `* ${s.indicator}: ${s.thisAvg.toFixed(1)}% ${s.unit} |` ) (${s.wow > 0 ? '+' : ''})
${s.wow > 0 ? '+' : ''}) % WoW) ${s.status}`.join("\n"); return `* 📊 주간 KPI 리포트 * \n \n ${top} \n \n * 하이라이
트 *: ${insight.highlight} \n * 우려 *: ${insight.concern} \n * 권고 *: ${insight.recommendation}`; } 강의 시연
포인트 KPI 시트의 한 셀을 일부러 큰 값으로 수정 → weeklyReport() 즉시 실행 → AI 코멘트가 "특정 지표 급등에 주
목" 등으로 바뀌는 모습 kpi_meta의 direction을 down으로 바꿔 "이탈을 같은 부정 지표도 같은 코드로 처리됨" 시연
임원에게 보낼 메일 미리보기로 "Excel 딱칠 보고서 vs 한 장 리포트" 비교 트러블슈팅 증상 원인 해결 WoW가 모두 0 일
자가 문자열로 들어옴 kpi_daily의 날짜 컬럼 형식을 "날짜"로 강제 AI 코멘트가 일반론 데이터 너무 적음 kpi_daily에
최소 14일치 데이터 누적 후 사용 트리거가 한 번도 안 돌아감 인증 미완료 Apps Script에서 한 번 수동 실행하여 권한
부여 일부 지표 누락 meta에 등록 안 됨 kpi_meta에 추가하지 않아도 표시되지만, 목표/상태가 비어 보임 응용 아이디
어 부서별 분기: 시트에 부서 컬럼 추가 → 부서별 메일을 다른 수신자에 발송 Looker Studio 연동: Apps Script가
Sheet에 적재한 결과를 Looker가 시각화 이상 감지 강화: 이번주 값이 표준편차 ±2σ 벗어나면 즉시 임원 알림 분리
OKR 연결: kpi_meta에 okr_id 컬럼 추가하여 OKR 트래킹과 통합

```

```

meta.target; return reached ? '✅ 달성' : '⚠️ 미달'; } function aiCommentary(summary) { const prompt
= `당신은 경영진 보고를 작성하는 시니어 분석가입니다. 다음 주간 KPI를 보고 한국어로 간결하게 작성하세요. 규칙: -
사실만, 추측 금지 - 5문장 이내 - 1) 가장 좋은 변화 1건 2) 가장 우려되는 변화 1건 3) 다음 주 권고 1건 - "~합니다"체
데이터: ${JSON.stringify(summary, null, 2)} JSON으로 반환:
{"highlight":"...", "concern":"...", "recommendation":"..."} `; const url = `https://
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${
GEMINI_KEY}`; const res = UrlFetchApp.fetch(url, { method:'post', contentType:'application/json',
payload: JSON.stringify({ contents:[{parts:[{text:prompt}]}], generationConfig:
{responseMimeType:'application/json'}} )); return
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
buildReportHTML(summary, insight, weekStart) { let rows = summary.map(s => { const arrow =
s.wow > 1 ? '▲' : s.wow < -1 ? '▼' : '▬'; const wowStr = s.wow ? `${s.wow.toFixed(1)}%` : '-';
return `<tr> <td style="padding:.5rem;border-bottom:1px solid #eee;">${s.indicator}</td> <td
style="padding:.5rem;border-bottom:1px solid #eee;text-align:right;">${s.thisAvg.toFixed(1)} $
{s.unit}||'`</td> <td style="padding:.5rem;border-bottom:1px solid #eee;text-align:right;">
{s.lastAvg.toFixed(1)}</td> <td style="padding:.5rem;border-bottom:1px solid #eee;text-
align:right;">${arrow} ${wowStr}</td> <td style="padding:.5rem;border-bottom:1px solid #eee;text-
align:right;">${s.target}||'—'</td> <td style="padding:.5rem;border-bottom:1px solid #eee;">
{s.status}</td> </tr>` ; }).join(''); return `<div style="font-family:'Noto Sans KR',sans-serif;max-
width:760px;line-height:1.6;color:#222;"> <h2 style="border-bottom:2px solid #b45309;padding-
bottom:.5rem;">주간 KPI 리포트</h2> <p style="color:#6a604f;">기준 주: ${fmt(weekStart)} ~ $
{fmt(new Date())}</p> <h3>🔴 핵심 코멘트</h3> <p><b>가장 좋은 변화 —</b>${insight.highlight}</p>
<p><b>우려 포인트 —</b>${insight.concern}</p> <p><b>다음 주 권고 —</b>
${insight.recommendation}</p> <h3 style="margin-top:1.5rem;">📊 지표별 표</h3> <table
style="width:100%;border-collapse:collapse;font-size:.95em;"> <thead
style="background:#fbf6ec;"> <tr><th style="padding:.5rem;text-align:left;">지표</th> <th
style="padding:.5rem;text-align:right;">이번주 평균</th> <th style="padding:.5rem;text-align:right;">
지난주 평균</th> <th style="padding:.5rem;text-align:right;">WoW</th> <th style="padding:.
5rem;text-align:right;">목표</th> <th style="padding:.5rem;text-align:left;">상태</th></tr> </thead>
<tbody>${rows}</tbody> </table> <p style="color:#999;font-size:.85em;margin-top:2rem;">자동 생성
· 원천 데이터: 회사 KPI 시트</p> </div>` ; } function buildSlack(summary, insight) { const top =
summary.map(s => `• ${s.indicator}: ${s.thisAvg.toFixed(1)}${s.unit}||'` ) (${s.wow>0?'+' ':'')
${s.wow.toFixed(1)}% WoW) ${s.status}` ).join('\n'); return `* 📊 주간 KPI 리포트* \n\n${top}\n\n*하이라이
트*: ${insight.highlight}\n*우려*: ${insight.concern}\n*권고*: ${insight.recommendation}` ; } 강의 시연
포인트 KPI 시트의 한 셀을 일부러 큰 값으로 수정 → weeklyReport() 즉시 실행 → AI 코멘트가 "특정 지표 급등에 주
목" 등으로 바뀌는 모습 kpi_meta의 direction을 down으로 바꿔 "이탈율 같은 부정 지표도 같은 코드로 처리됨" 시연
임원에게 보낼 메일 미리보기로 "Excel 딱칠 보고서 vs 한 장 리포트" 비교 트러블슈팅 증상 원인 해결 WoW가 모두 0 일
자가 문자열로 들어옴 kpi_daily의 날짜 컬럼 형식을 "날짜"로 강제 AI 코멘트가 일반론 데이터 너무 적음 kpi_daily에
최소 14일치 데이터 누적 후 사용 트리거가 한 번도 안 돌아감 인증 미완료 Apps Script에서 한 번 수동 실행하여 권한
부여 일부 지표 누락 meta에 등록 안 됨 kpi_meta에 추가하지 않아도 표시되지만, 목표/상태가 비어 보임 응용 아이디
어 부서별 분기: 시트에 부서 컬럼 추가 → 부서별 메일을 다른 수신자에 발송 Looker Studio 연동: Apps Script가
Sheet에 적재한 결과를 Looker가 시각화 이상 감지 강화: 이번주 값이 표준편차 ±2σ 벗어나면 즉시 임원 알림 분리
OKR 연결: kpi_meta에 okr_id 컬럼 추가하여 OKR 트래킹과 통합

```



```

meta.target; return reached ? '✅ 달성' : '⚠️ 미달'; } function aiGeneratory(summary) { const prompt
= `당신은 경영진 보고를 작성하는 시니어 분석가입니다. 다음 주간 KPI를 보고 한국어로 간결하게 작성하세요. 규칙: -
사실만, 추측 금지 - 5문장 이내 - 1) 가장 좋은 변화 1건 2) 가장 우려되는 변화 1건 3) 다음 주 권고 1건 - "~합니다"체
데이터: ${JSON.stringify(summary, null, 2)} JSON으로 반환:
{"highlight":"...", "concern":"...", "recommendation":"..."} `; const url = `https://
})gjoin(itV)language.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=$
{GEMINI_KEY}`; const res = UrlFetchApp.fetch(url, { method:'post', contentType:'application/json',
return `

## 강의 시연 포인트



1. KPI 시트의 한 셀을 일부러 큰 값으로 수정 → weeklyReport() 즉시 실행 → AI 코멘트가 "특정 지표 급등에 주목" 등으로 바뀌는 모습
2. kpi_meta의 direction을 down으로 바꿔 "이탈율 같은 부정 지표도 같은 코드로 처리됨" 시연
3. 임원에게 보낼 메일 미리보기로 "Excel 딱칠 보고서 vs 한 장 리포트" 비교



22 / 68


```


네다바웨이 · WORKBOOK

HR 자동화 (3선)

설정 가이드

[illegible]

마지막 컬럼을 비워두세요. 스크립트가 완료 표시

1. 환영 레터 ({{company_name}}, {{company_address}}, {{company_phone}}, {{company_email}} 변수 포함)

Step 3. Apps Script 속성

WELCOME TEMPLATE ID

왜 설치형인가: 단순 onEdit 은 권한 한도가 낮아 GmailApp.sendEmail ,

스크립트 상단의 **installTrigger** 함수를 한 번 실행 → 권한 동의 화면이 뜨면 모두 허용

26 / 68


```

+ 체크리스트 Doc 생성 const welcomeDoc = copyAndFill(TPL_WEL, `[환영] ${name} 님 - ${fmt(joinDate)}
`, { 이름: name, 입사일: fmt(joinDate), 부서: dept, 직책: role, 멘토: mentor || '추후 안내' }); const
checklistDoc = copyAndFill(TPL_CHK, `[체크리스트] ${name} 90일 플랜`, { 이름: name, 입사일:
fmt(joinDate), 부서: dept, 멘토: mentor || '' }); // 2) 환영 메일 (입사자 + 멘토 cc) const body =
buildWelcomeBody(name, joinDate, dept, role, mentor, welcomeDoc.url, checklistDoc.url);
GmailApp.sendEmail(email, `${name} 님, ${fmt(joinDate)} 입사를 환영합니다`, body, { name:
FROM_NAME, cc: mentorEmail || '', htmlBody: body }); // 3) 캘린더 — Day 1 환영미팅 + Day 7·30·90 체크
인 자동 생성 const cal = CalendarApp.getDefaultCalendar(); const join = new Date(joinDate);
cal.createEvent(`${name} 환영 미팅`, atHour(join, 10), atHour(join, 11), { guests: `${email},${
mentorEmail}||` }); [7, 30, 90].forEach(d => { const day = addDays(join, d); cal.createEvent(`${
name} ${d}일 체크인`, atHour(day, 14), atHour(day, 14, 30), { guests: `${email},${mentorEmail}||`
}); }); // 4) 슬랙 공지 if (SLACK) { UrlFetchApp.fetch(SLACK, { method: 'post', contentType:
'application/json', payload: JSON.stringify({ text: `👋 *신규 입사 안내* \n*${name}* 님이 $
{fmt(joinDate)} *${dept}/${role}* 으로 합류합니다.\n멘토: ${mentor}||'미정'`\n환영 한 마디 남겨주세
요.` }); }); // 5) 상태 업데이트 sheet.getRange(row, 8).setValue('완료'); sheet.getRange(row,
9).setValue(welcomeDoc.url); sheet.getRange(row, 10).setValue(checklistDoc.url); } function
copyAndFill(templateId, title, vars) { const folder = DriveApp.getFolderById(FOLDER); const file =
DriveApp.getFileById(templateId).makeCopy(title, folder); const doc =
DocumentApp.openById(file.getId()); const body = doc.getBody(); for (const k in vars)
{ body.replaceText(`{{${k}}}`, String(vars[k])); } doc.saveAndClose(); return { id: file.getId(), url:
file.getUrl() }; } function buildWelcomeBody(name, joinDate, dept, role, mentor, welcomeUrl,
checklistUrl) { return `<div style="font-family:'Noto Sans KR',sans-serif;line-height:1.7;max-width:
620px;"><h2 style="color:#b45309;">${name} 님, 환영합니다 🙌</h2><p>${fmt(joinDate)} ${dept}/${
role}로 입사하시는 ${name} 님께 첫날을 위한 안내를 드립니다.</p><ul><li>📄 <a href="${welcomeUrl}">환
영 레터(개인 맞춤)</a></li><li>✅ <a href="${checklistUrl}">90일 온보딩 체크리스트</a></li><li>👤 멘토:
${mentor || '추후 배정'}</li><li>📅 첫날 10:00 환영 미팅이 캘린더에 등록되었습니다.</li></ul><p>입사 전 궁
금한 점은 언제든지 회신주세요. 좋은 인연이 되길 기대합니다.</p><p style="color:#6a604f;">— ${FROM_NAME}</p></div>`; } function fmt(d) { return new Date(d).toISOString().slice(0,10); } function atHour(d, h,
m=0) { const x = new Date(d); x.setHours(h,m,0,0); return x; } function addDays(d, n) { const x = new
Date(d); x.setDate(x.getDate()+n); return x; } 강의 시연 포인트 시트에 입사자 한 줄을 시연용으로 입력 → 캘린
더·메일·슬랙·드라이브가 동시에 반응 Docs 템플릿의 {{이름}} 같은 변수가 어떻게 치환되는지 강조 "왜 이 작업은 사람이
해야 하는가" 토론: 멘토 매칭·문화 안내 영상은 자동화 대상에서 의도적으로 제외했음을 설명 트러블슈팅 증상 원인 해결
onEdit 미발동 / 권한 오류 단순 트리거는 외부 서비스 호출 권한이 없음 셋업 Step 4에 따라 installTrigger()를 1회
실행해 설치형 트리거로 등록 캘린더 이벤트 중복 생성 같은 행을 다시 편집 status 컬럼이 완료면 조기 return — 이미
처리된 환영 메일 스팸 분류 발신 도메인/SPF 미설정 Workspace 도메인 발신 + SPF/DKIM 설정 멘토 미정 시 캘린더
오류 guests 빈값 mentorEmail이 빈문자열일 때 분기 처리(이미 코드에 반영) 응용 아이디어 계정 신청 연동: 입사일
-7일에 IT팀 채널로 계정 신청 카드 발송 온보딩 진행률: 체크리스트 Doc의 체크 항목을 주 1회 스캔해 시트에 진행률 적
재 퇴사자 오프보딩 미리: 같은 패턴으로 퇴사 절차(자료 인계·계정 정리) 자동화 다국어: 외국인 직원의 경우 환영 레터를
영어 템플릿으로 분기

```



```
[meta[0]]|payload.channel.id|$[meta[1]]|payload.msg|$[user.Name]`); // 즉시 Slack에 message 교체 응답 — chat.update API 호출 불필요 (왕복 시간 절약) const resultText = decided === '승인' ? `✓ ${userName} 님이 ${name}의 ${type} *승인* 했습니다.` : `✗ ${userName} 님이 ${name}의 ${type} *반려* 했습니다.`; return jsonResponse({ replace_original: true, text: resultText, blocks: [ ...payload.message.blocks.filter(b => b.type !== 'actions'), {type:'context', elements:[{type:'mrkdwn', text:resultText+' (후속 처리 진행 중)'}]}] } ); function jsonResponse(obj) { return ContentService.createTextOutput(JSON.stringify(obj)).setMimeType(ContentService.MimeType.JSON); } /** * 설치용 onEdit 트리거: 폼 응답 시트의 마지막 컬럼이 `...|승인` 또는 `...|반려`로 * 바뀌면 캘린더·연차잔여·신청자 메일을 처리한다. 셋업 시 `installApprovalTrigger()`를 1회 실행. */ function onApprovalStatusChange(e) { if (!e || !e.range) return; const sheet = e.range.getSheet(); // 트리거는 폼 응답 시트(첫 시트)의 메타 컬럼 변화만 처리한다. // 시트 인덱스 비교(getIndex === 1)가 이를 비교보다 가벼워 추가 openById 호출이 불필요하다. if (sheet.getIndex() !== 1) return; const lastCol = sheet.getLastColumn(); if (e.range.getColumn() !== lastCol) return; // 메타 컬럼만 감시 const r = e.range.getRow(); if (r === 1) return; const meta = String(e.value || '').split('|'); const decided = meta[2]; if (decided !== '승인' && decided !== '반려') return; // 처리 역등성: 4번째 토큰("done")이 이미 있으면 skip if (meta[4] === 'done') return; const row = sheet.getRow(r); const [ts, email, name, type, start, end, reason, leadEmail] = row; const days = calcDays(type, start, end); if (decided === '승인') { CalendarApp.getCalendarById(CAL_ID).createNewDayEvent(`${name} ${type}`, new Date(start), new Date(end)); } else { notifyApplicant(email, `✗ ${type} 신청이 반려되었습니다. 팀장과 협의해주세요.`); } // done 마킹으로 역등성 보장 sheet.getRange(r, lastCol).setValue(`${meta[0]}|${meta[1]}|${decided}|${meta[3]}||done`); } /** * 셋업 시 1회 실행: onApprovalStatusChange를 설치형 onEdit 트리거로 등록. * 단순 트리거는 GmailApp/CalendarApp 호출 권한이 없으므로 반드시 설치형이어야 한다. */ function installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); ScriptApp.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange').forEach(t => ScriptApp.deleteTrigger(t)); ScriptApp.newTrigger('onApprovalStatusChange').forSpreadsheet(ss).onEdit().create().Logger.log('승인 상태 변경 트리거 등록 완료'); } // 헬퍼 //
```

Signature 헤더와 Slack Signing Secret 으로 HMAC 검증 을 추가하세요. 검증 패턴은 가이드 끝 "응용 아이디어"에 요약되어 있습니다.

셋업 가이드

Step 1. Google Form

필수 질문: 1. 이름 (단답) 2. 직위 이메일 (단답) "응답을 기록할 수 있도록 이메일 수집"으로 자동 채움 권장) 3. 종류 (객관식 연차, 반차 오전, 반차 오후, 재택) 4. 시작일 (날짜) 5. 종료일 (날짜) 6. 사유 (장문) 7. 팀장이 메일 (단답) 응답 → 시트로 연결

Step 2. 시트 추가 탭 만들기

balances: 이메일 | 연초잔고 | 사용 | 잔여 팀 lead_email: 팀장이메일 | slack_user_id (Slack 멤버 ID; @팀장 → 옵션 → 멤버 ID 복사)

Step 3. Slack App 설정

1. Bot Token Scopes: chat:write, chat:write.public — email users:read, users:read.email 2. Interactivity & Shortcuts ON → Request URL (Apps Script Web App URL — Step 5에서 받음) 3. Bot User OAuth Token 복사 → Apps Script 속성 SLACK_BOT_TOKEN

Step 4. Apps Script 속성 등록

• RESPONSE SHEET ID CALENDAR_ID (전사 캘린더 ID: 캘린더 설정에서 복사) • SLACK_BOT_TOKEN 시나리오: 연차 잔여를 0으로 미리 세팅한 직원의 신청이 자동 차단되는 모습 시연 트러블슈팅 증상 원인 해결 슬랙 카드 미수신 Bot이 채널/DM에 미초대 팀장 DM은 users:read 후 slackUid를 정확히 (U... 형식) doPost가 동작 안 함 Web App 배포가 불안한 것으로 팀 내부로 시연 배포, 새 URL을 Slack App에 갱신 캘린더 권한 오류 CALENDAR_ID가 비공개 캘린더 공유 → 스크립트 실행 계정에 변경 권한 부여 Slack에 "took too long to respond" 표시 doPost가 3초 안에 응답 못 함

Step 5. 트리거 + Web App 배포

1. 트리거 1: onFormSubmit → 폼 제출 시 실행 (Apps Script 트리거 메뉴에서 추가) 2. 트리거 2: onEdit 트리거를 Install Approval Trigger 함수를 1회 실행 → 연한 동작 후 설치형 onEdit 트리거가 등록됨 (단순 트리거가 아니어야 GmailApp/CalendarApp 호출 가능) 3. 배포 → 새 배포 → 웹 앱 실행: 본인 / 애셋: 누구나 → URL 복사 → Slack App의 Interactivity Request URL에 붙여넣기

SLACK_SIGNING_SECRET에 저장한 뒤, 다음 가드를 코드 최상단에 추가합니다. function verifySlackSignature(e) { const secret = PROPS.getProperty('SLACK_SIGNING_SECRET'); if (!secret) return true; // 셋업 안 했으면 검증 생략 (개발용) const ts = e.parameter['X-Slack-Request-Timestamp'] || e.headers && e.headers['X-Slack-Request-Timestamp']; const sig = e.parameter['X-Slack-Signature'] || e.headers && e.headers['X-Slack-Signature']; if (!ts || !sig) return false; // 5분 이상 지난 요청은 재전송 공격 가능성으로 거부 if (Math.abs(Date.now()/1000 - Number(ts)) > 300) return false; const base = `v0:\${ts}:\${e.postData && e.postData.contents || ''}`; const mac = Utilities.computeHmacSha256Signature(base, secret).map(b => ('0' + (b & 0xff).toString(16)).slice(-2)).join(''); return `v0:\${mac}` === sig; } Apps Script Web App은 표준 헤더 접근에 제한이 있어 Slack이 보내는 서명 정보를 캡처하려면 별도 프록시(Cloud Functions, Cloudflare Worker 등)를 두는 패턴이 가장 안전합니다. 강의에서는 "왜 검증이 필요한지·어디서 막을지"를 토론 주제로 다루고, 실제 코드 추가는

```

{meta[0]][payload.channel.id]}[${meta[1]][payload.message.ts]}[${payload.message}`); // 즉시 Slack에
message 교체 응답 — chat.update API 호출 불필요 (왕복 시간 절약) const resultText = decided === '승인' ? `✓
${userName} 님이 ${name}의 ${type} *승인* 했습니다.` : `✗ ${userName} 님이 ${name}의 ${type} *반려* 했
습니다.`; return jsonResponse({ replace_original: true, text: resultText, blocks:
[ ...payload.message.blocks.filter(b => b.type !== 'actions'), {type:'context', elements:[{type:'mrkdwn',
text: resultText + ' (후속 처리 진행 중)'}]}] }); } function jsonResponse(obj) { return
ContentTypes.createEventPayload(Slack.SlackEventPayload, obj); } } // 메타 컬럼의 변경이 발생하면 (onApprovalStatusChange); }
/** * 설치형 onEdit 트리거: 폼 응답 시트의 마지막 컬럼이 '승인' 또는 '반려'로 * 바뀌면 캘린더 연차잔여, 신청자
메일을 처리한다. * 셋업 시 — installApprovalTrigger() 를 1회 실행. */ function onApprovalStatusChange(e) { if
(!e || !e.range) return; const sheet = e.range.getSheet(); // 트리거는 폼 응답 시트(첫 시트)의 메타 컬럼 변화만 처
리한다. // 시트 인덱스 비교(getIndex === 1)가 이를 비교보다 가벼워 추가 openById 호출이 불필요하다. if
(sheet.getIndex() !== 1) return; const lastCol = sheet.getLastColumn(); if (e.range.getColumn() !==
lastCol) return; // 메타 컬럼만 감지 const r = e.range.getRow(); if (r === 1) return; const meta =
String(e.value || '').split('|'); const decided = meta[2]; if (decided !== '승인' && decided !== '반려')
return; // 처리 역등성: 4번째 토큰("done")이 이미 있으면 skip if (meta[4] === 'done') return; const row =
sheet.getRange(r, 1, 1, lastCol).getValues()[0]; const [ts, email, name, type, start, end, reason,
leadEmail] = row; const days = calcDays(type, start, end); if (decided === '승인')
{ CalendarApp.getCalendarById(CAL_ID).createAllDayEvent( `${name} ${type}`, new Date(start), new
Date(new Date(end).getTime()+86400000) ); if (type === '연차') updateBalance(email, days);
notifyApplicant(email, `✓ ${type} 승인되었습니다. (${fmt(start)} ~ ${fmt(end)})`); } else
{ notifyApplicant(email, `✗ ${type} 신청이 반려되었습니다. 팀장과 협의해주세요.`); } // done 마킹으로 역등성 보
장 sheet.getRange(r, lastCol).setValue(`${meta[0]}|${meta[1]}|${decided}|${meta[3]}||done`); } /** * 셋
업 시 1회 실행: onApprovalStatusChange를 설치형 onEdit 트리거로 등록. * 단순 트리거는 GmailApp/
CalendarApp 호출 권한이 없으므로 반드시 설치형이어야 한다. */ function installApprovalTrigger() { const ss =
SpreadsheetApp.openById(SHEET_ID); ScriptApp.getProjectTriggers().filter(t => t.getHandlerFunction()
=== 'onApprovalStatusChange').forEach(t => ScriptApp.deleteTrigger(t));
ScriptApp.newTrigger('onApprovalStatusChange').forSpreadsheet(ss).onEdit().create(); Logger.log('승인
상태 변경 트리거 등록 완료'); } //
----- // 헬퍼 //
----- function
calcDays(type, start, end) { if (type.indexOf('반차') >= 0) return 0.5; if (!end) return 1; const ms = new
Date(end) - new Date(start); return Math.max(1, Math.round(ms / 86400000) + 1); } function
getBalance(email) { const sh = SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances');
const data = sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) { if (data[i][0] === email)
{ return {row: i+1, initial: data[i][1], used: data[i][2], remaining: data[i][3]}; } } return null; } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances'); sh.getRange(bal.row,
3).setValue((bal.used||0) + days); sh.getRange(bal.row, 4).setValue((bal.initial||0) - ((bal.used||
0)+days)); } function lookupTeamLeadSlack(leadEmail) { const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('team_lead_slack'); const data =
sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) if (data[i][0] === leadEmail) return
data[i][1]; return null; } function notifyApplicant(email, text) { GmailApp.sendEmail(email, '[휴가 신청 알
림]', text); } function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/$
{method}`, { method: 'post', contentType: 'application/json; charset=utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); } 강의 시연 포인트 강사가 폼 제출 → 30초 내 팀
장(강사 본인) Slack에 카드 도착 "승인" 버튼 클릭 → 캘린더·시트가 동시 갱신, Slack 카드 본문도 즉시 업데이트 잔여 부족
시나리오: 연차 잔여를 0으로 미리 세팅한 직원의 신청이 자동 차단되는 모습 시연 트러블슈팅 증상 원인 해결 슬랙 카드 미수
신 Bot이 채널/DM에 미초대 팀장 DM은 users:read 후 slackUid를 정확히 (U... 형식) doPost가 동작 안 함 Web App
배포가 "본인만"으로 됨 "누구나"로 재배포, 새 URL을 Slack App에 갱신 캘린더 권한 오류 CALENDAR_ID가 비공개 캘린
더 공유 → 스크립트 실행 계정에 변경 권한 부여 Slack에 "took too long to respond" 표시 doPost가 3초 안에 응답 못
함 본 가이드 코드는 무거운 작업을 설치형 onEdit 트리거로 분리해 doPost를 0.5~1초 수준으로 유지함.
installApprovalTrigger가 등록되어 있는지 확인 같은 신청을 두 번 처리 메시지 새로고침 후 재클릭 sheet 메타 컬럼의 역
등성 가드('대기' 외 상태면 무시)가 자동으로 차단함 응용 아이디어 2단계 승인: 팀장 → 임원 두 단계가 필요한 경우 state 컬
럼으로 단계 관리 연차 자동 적립: 매월 1일 트리거로 balances에 비례 적립 자동 추가 공휴일 보정: calcDays에서 공공
API(공휴일 RSS)로 주말·공휴일 제외 이상 신청 감지: 같은 직원이 단기간 반복 신청할 때 HR에 부드러운 알림 프로덕션 보
강: Slack 서명 검증 (HMAC) 운영 환경에서는 외부 위조 페이로드를 차단하기 위해 doPost 진입 직후 Slack 서명 검증을
수행하세요. Slack App 관리 화면 → Basic Information → Signing Secret을 발급받아 스크립트 속성
SLACK_SIGNING_SECRET에 저장한 뒤, 다음 가드를 코드 최상단에 추가합니다. function verifySlackSignature(e)
{ const secret = PROPS.getProperty('SLACK_SIGNING_SECRET'); if (!secret) return true; // 셋업 안 했으면
검증 생략 (개발용) const ts = e.parameter['X-Slack-Request-Timestamp'] || e.headers && e.headers['X-
Slack-Request-Timestamp']; const sig = e.parameter['X-Slack-Signature'] || e.headers && e.headers['X-
Slack-Signature']; if (!ts || !sig) return false; // 5분 이상 지난 요청은 재전송 공격 가능성으로 거부 if
(Math.abs(Date.now()/1000 - Number(ts)) > 300) return false; const base = `v0:${ts}:${e.postData &&
e.postData.contents || ''}`; const mac = Utilities.computeHmacSha256Signature(base, secret) .map(b =>
('0' + (b & 0xff).toString(16)).slice(-2)).join(''); return `v0:${mac}` === sig; } Apps Script Web App은 표준
헤더 접근에 제한이 있어 Slack이 보내는 서명 정보를 캡처하려면 별도 프록시(Cloud Functions, Cloudflare Worker
등)를 두는 패턴이 가장 안전합니다. 강의에서는 "왜 검증이 필요한지·어디서 막을지"를 토론 주제로 다루고, 실제 코드 추가는

```

```

{meta[0]][payload.channel.id]}[${meta[1]][payload.msgs]}{decided}`}${userName}`); // 즉시 Slack에
message 교체 응답 — chat.update API 호출 불필요 (왕복 시간 절약) const resultText = decided === '승인' ? `✅
${userName} 님이 ${name}의 ${type} *승인* 했습니다.` : `❌ ${userName} 님이 ${name}의 ${type} *반려* 했
습니다.`; return jsonResponse({ replace_original: true, text: resultText, blocks:
[ ...payload.message.blocks.filter(b => b.type !== 'actions'), {type:'context', elements:[{type:'mrkdwn',
text: resultText + ' (후속 처리 진행 중)'}]} ] }); } function jsonResponse(obj) { return
ContentService.createTextOutput(JSON.stringify(obj)) .setMimeType(ContentService.MimeType.JSON); }
/* * 설치형 onEdit 트리거. 폼 응답 시트의 마지막 컬럼이 `...|승인` 또는 `...|반려`로 * 바뀌면 캘린더·연차잔여·신청자
메일을 처리한다. 셋업 시 `installApprovalTrigger()`를 1회 실행. */ function onApprovalStatusChange(e) { if
(!e || !e.range) return; const sheet = e.range.getSheet(); // 트리거는 폼 응답 시트(첫 시트)의 메타 컬럼 변화만 처
리한다. // 시트 인덱스 비교(getIndex === 1)가 이름 비교보다 가벼워 추가 openById 호출이 불필요하다. if
(sheet.getIndex() !== 1) return; const lastCol = sheet.getLastColumn(); if (e.range.getColumn() !==
lastCol) return; // 메타 컬럼만 감시 const r = e.range.getRow(); if (r === 1) return; const meta =
String(e.value || '').split('|'); const decided = meta[2]; if (decided !== '승인' && decided !== '반려')
return; // 처리 역동성: 4번째 토큰("done")이 이미 있으면 skip if (meta[4] === 'done') return; const row =
sheet.getRange(r, 1, 1, lastCol).getValues()[0]; const [ts, email, name, type, start, end, reason,
leadEmail] = row; const days = calcDays(type, start, end); if (decided === '승인')
{ CalendarApp.getCalendarById(CAL_ID).createAllDayEvent(` ${name} ${type}`, new Date(start), new
Date(new Date(end).getTime()+86400000) ); if (type === '연차') updateBalance(email, days);
notifyApplicant(email, `✅ ${type} 승인되었습니다. (${fmt(start)} ~ ${fmt(end)})`); } else
{ notifyApplicant(email, `❌ ${type} 신청이 반려되었습니다. 팀장과 협의해주세요.`); } // done 마킹으로 역동성 보
장 sheet.getRange(r, lastCol).setValue(` ${meta[0]}|${meta[1]}|${decided}|${meta[3]}|'|done`); } /* * 셋
업 시 1회 실행: onApprovalStatusChange를 설치형 onEdit 트리거로 등록. * 단순 트리거는 GmailApp/
CalendarApp 호출 권한이 없으므로 반드시 설치형이어야 한다. */ function installApprovalTrigger() { const ss =
SpreadsheetApp.openById(SHEET_ID); ScriptApp.getProjectTriggers().filter(t => t.getHandlerFunction()
=== 'onApprovalStatusChange').forEach(t => ScriptApp.deleteTrigger(t));
ScriptApp.newTrigger('onApprovalStatusChange').forSpreadsheet(ss).onEdit().create(); Logger.log('승인
상태 변경 트리거 등록 완료'); } //
----- // 헬퍼 //
----- function
calcDays(type, start, end) { if (type.indexOf('반차') >= 0) return 0.5; if (!end) return 1; const ms = new
Date(end) - new Date(start); return Math.max(1, Math.round(ms / 86400000) + 1); } function
getBalance(email) { const sh = SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances');
const data = sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) { if (data[i][0] === email)
{ return {row: i+1, initial: data[i][1], used: data[i][2], remaining: data[i][3]}; } } return null; } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances'); sh.getRange(bal.row,
3).setValue((bal.used||0) + days); sh.getRange(bal.row, 4).setValue((bal.initial||0) - ((bal.used||
0)+days)); } function lookupTeamLeadSlack(leadEmail) { const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('team_lead_slack'); const data =
sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) if (data[i][0] === leadEmail) return
data[i][1]; return null; } function notifyApplicant(email, text) { GmailApp.sendEmail(email, '[휴가 신청 알
림]', text); } function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${
method}`, { method: 'post', contentType: 'application/json; charset=utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); } 강의 시연 포인트 강사가 폼 제출 → 30초 내 팀
장(강사 본인) Slack에 카드 도착 "승인" 버튼 클릭 → 캘린더·시트가 동시 갱신, Slack 카드 본문도 즉시 업데이트 잔여 부족
시나리오: 연차 잔여를 0으로 미리 세팅한 직원의 신청이 자동 차단되는 모습 시연 트러블슈팅 증상 원인 해결 슬랙 카드 미수
신 Bot이 채널/DM에 미초대 팀장 DM은 users:read 후 slackUid를 정확히 (U... 형식) doPost가 동작 안 함 Web App
배포가 "본인만"으로 됨 "누구나"로 재배포, 새 URL을 Slack App에 갱신 캘린더 권한 오류 CALENDAR_ID가 비공개 캘린
더 공유 → 스크립트 실행 계정에 변경 권한 부여 Slack에 "took too long to respond" 표시 doPost가 3초 안에 응답 못
함 본 가이드 코드는 무거운 작업을 설치형 onEdit 트리거로 분리해 doPost를 0.5~1초 수준으로 유지함.
installApprovalTrigger가 등록되어 있는지 확인 같은 신청을 두 번 처리 메시지 새로고침 후 재클릭 sheet 메타 컬럼의 역
동성 가드('대기' 외 상태면 무시)가 자동으로 차단함 응용 아이디어 2단계 승인: 팀장 → 임원 두 단계가 필요한 경우 state 컬
럼으로 단계 관리 연차 자동 적립: 매월 1일 트리거로 balances에 비례 적립 자동 추가 공휴일 보정: calcDays에서 공공
API(공휴일 RSS)로 주말·공휴일 제외 이상 신청 감지: 같은 직원이 단기간 반복 신청할 때 HR에 부드러운 알림 프로덕션 보
강: Slack 서명 검증 (HMAC) 운영 환경에서는 외부 위조 페이로드를 차단하기 위해 doPost 진입 직후 Slack 서명 검증을
수행하세요. Slack App 관리 화면 → Basic Information → Signing Secret을 발급받아 스크립트 속성
SLACK_SIGNING_SECRET에 저장한 뒤, 다음 가드를 코드 최상단에 추가합니다. function verifySlackSignature(e)
{ const secret = PROPS.getProperty('SLACK_SIGNING_SECRET'); if (!secret) return true; // 셋업 안 했으면
검증 생략 (개발용) const ts = e.parameter['X-Slack-Request-Timestamp'] || e.headers && e.headers['X-
Slack-Request-Timestamp']; const sig = e.parameter['X-Slack-Signature'] || e.headers && e.headers['X-
Slack-Signature']; if (!ts || !sig) return false; // 5분 이상 지난 요청은 재전송 공격 가능성으로 거부 if
(Math.abs(Date.now()/1000 - Number(ts)) > 300) return false; const base = `v0:${ts}:${e.postData &&
e.postData.contents || ''}`; const mac = Utilities.computeHmacSha256Signature(base, secret) .map(b =>
('0' + (b & 0xff).toString(16)).slice(-2)).join(''); return `v0:${mac}` === sig; } Apps Script Web App은 표준
헤더 접근에 제한이 있어 Slack이 보내는 서명 정보를 캡처하려면 별도 프록시(Cloud Functions, Cloudflare Worker
등)를 두는 패턴이 가장 안전합니다. 강의에서는 "왜 검증이 필요한지·어디서 막을지"를 토론 주제로 다루고, 실제 코드 추가는

```


[illegible]

```

{meta[0]||payload.channel.id}||payload.message.ts}||$decided}||$user_name`); // 즉시 Slack에
message 교체 응답 — chat.update API 호출 불필요 (왕복 시간 절약) const resultText = decided === '승인' ? `
  ${user_name}님이 ${name}의 ${type} *승인* 했습니다.` : `
  ✖️ ${user_name}님이 ${name}의 ${type} *반려* 했
  습니다.`; return jsonResponse({ replace_original: true, text: resultText, blocks:
  [ ...payload.message.blocks.filter(b => b.type !== 'actions'), {type:'context', elements:[{type:'mrkdwn',
  }
  text: resultText + ' (후속 처리 진행 중)'}] }]); } function jsonResponse(obj) { return
ContentService.createTextOutput(JSON.stringify(obj)) .setMimeType(ContentService.MimeType.JSON); }
/** * 설치형 onEdit 트리거. 폼 응답 사트의 마지막 컬럼에 `...[승인]` 또는 `...[반려]`로 * 바뀌면 캘린더·연차잔여·신청자
메일을/저2)한Slack인터랙션콜백·버튼눌림 처리랑()`를 1회 실행. */ function onApprovalStatusChange(e) { if
(!e || !e.range) return; const sheet = e.range.getSheet(); // 트리거는 폼 응답 시트(첫 시트)의 메타 컬럼 변화만 처
// Slack은 Interactive payload에 대한 HTTP 응답을 3초 안에 받아 못하면 가 openByld 호출이 불필요하다. if
(// `This app took too long to respond` 오류를 노출한다. 따라서 doPost는 if (e.range.getColumn() !==
// (a) 빠른 sheet 상태 갱신만 감행하고 (b) 즉시 Slack에 message 교체) 응답을 === 1) return; const meta =
// 2) 반환한다. a) 캘린더 '등록·연차 잔여 갱신·신청자 메일'은 sheet 2상태 컬럼을 ded !== '승인' && decided !== '반려'
// 3) 감시·하는 설치형 onEdit 트리거("onApprovalStatusChange")에서 비용제로 == 'done') return; const row =
// 처리한다. a) 분리가, 라이브 시연 안정성의 핵심이다[0]; const [ts, email, name, type, start, end, reason,
// ————— leadEmail] = row; const days = calcDays(type, start, end); if (decided === '승인')
{ const doPost = (calByld(CAL_ID).createAllDayEvent(` ${name} ${type}`, new Date(start), new
const payload = JSON.parse(e.parameter['payload']); (type === '연차') updateBalance(email, days);
const action = payload.actions[0].value; if (type === '승인' || type === '반려') {
const value = JSON.parse(action.value); if (type === '승인') {
const user_name = payload.user_name; a[0]}||$meta[1]}||$decided}||$meta[3]||`"done"); } /** * 셋
업 시 1회 실행: onApprovalStatusChange를 설치형 onEdit 트리거로 등록. * 단순 트리거는 GmailApp/
Calendar 스프레드시트 App.openByld(SHEET_ID); */ function installApprovalTrigger() { const ss =
SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId =
ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction()
const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인
ScriptApp.deleteTrigger(statusChange); } // 상태 변경 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
sheet.getRange(1, 1, 1, 4).setValues([bal.initial, bal.used, bal.remaining, bal.remaining]); } function
installApprovalTrigger() { const ss = SpreadsheetApp.openById(SHEET_ID); const sheet = ss.getSheetByName('balances'); const sheetId = ss.getId(); const sheetByld = ss.getSheetByld(sheetId); const triggers = sheetByld.getProjectTriggers().filter(t => t.getHandlerFunction() === 'onApprovalStatusChange'); const lastCol = sheet.getLastColumn(); const statusChange = sheet.getRange(1, lastCol, 1, lastCol).getValues()[0][0]; create(); Logger.log('승인 트리거 등록 완료'); } // 헬퍼 //
// 맥등성 가드: 이미 처리된 행이면 무시 (사용자가 메시지를 새로고쳐 두 번 누르는 경우 방지) function
calcDays(type, start, end) { if (type === '연차') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } if (type === '공휴일') { return Math.max(1, Math.floor((end.getTime() - start.getTime()) / (1000 * 60 * 60 * 24)) + 1); } } function
getBalance(email, days) { const bal = getBalance(email); if
```



```
{meta[0]]|payload.message.channel.id|$meta[1]]|payload.message.msgts|$decided|$userName)`); // 즉시 Slack에
message 교체 응답 - chat.update API 호출 불필요 (왕복 시간 절약) const resultText = decided === '승인' ? `✓
${userName} 님이 ${name}의 ${type} *승인* 했습니다.` : `✗ ${userName} 님이 ${name}의 ${type} *반려* 했
습니다.`; return jsonResponse({ replace_original: true, text: resultText, blocks:
[ ...payload.message.blocks.filter(b => b.type !== 'actions'), {type:'context', elements:[{type:'mrkdwn',
if (!end) return 1; resultText + ' (후속 처리 진행 중)'}] ] }); } function jsonResponse(obj) { return
ContentService.new Date(end)) JS new Date(start)) .setMimeType(ContentService.MimeType.JSON); }
/** * 설치 후 1초 (Math.max(1, Math.round(ms / 86400000) |승인); 또는 ...|반려'로 * 바뀌면 캘린더·연차잔여·신청자
메일을 처리한다. 셋업 시 `installApprovalTrigger()`를 1회 실행. */ function onApprovalStatusChange(e) { if
(!e) { function getBalance(email) { e.range.getSheet(); // 트리거는 폼 응답 시트(첫 시트)의 메타 컬럼 변화만 처
리한다. const sh = SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances'); id 호출이 불필요하다. if
(sh const data = sh.getDataRange().getValues(); sheet.getLastColumn(); if (e.range.getColumn() !===
for (let i = 1; i < data.length; i++) { const r = e.range.getRow(); if (r === 1) return; const meta =
String(data[i][0]) === email) { const decided = meta[2]; if (decided !== '승인' && decided !== '반려')
return; return {row: 4, initial: data[i][1], used: data[i][2], remaining: data[i][3]};
sheet.getRange(r, 1, 1, lastCol).getValues()[0]; const [ts, email, name, type, start, end, reason,
]
leadEmail] = row; const days = calcDays(type, start, end); if (decided === '승인')
{ CalendarApp.CalendarById(CAL_ID).createAllDayEvent(` ${name} ${type}`, new Date(start), new
Date(new Date(end).getTime()+86400000) ); if (type === '연차') updateBalance(email, days);
function updateBalance(email, days) { const bal = getBalance(email); if (bal) return;
const sh = SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances');
sh.getRange(bal.row, 3).setValue((bal.used+days)); 트리거로 등록. * 단순 트리거는 GmailApp/
CalendarApp 등을 관할할 필요 없이, 반드시 신청해야 하므로 function onUnlimitedApprovalStatusChange(e) { const ss =
SpreadsheetApp.openById(SHEET_ID); ScriptApp.getProjectTriggers().filter(t => t.getHandlerFunction()
function lookupTeamLeadSlack(leadEmail) {
ScriptApp.newTrigger('teamLeadSlackStatusChange').for(SpreadsheetApp.openById(CAL_ID)).create().trigger.log('승인
상태 변경 트리거 등록 완료'); } //
for (let i = 1; i < data.length; i++) if (data[i][0] === leadEmail) return data[i][1];
return null;
function calcDays(type, start, end) { if (type.indexOf('반차') >= 0) return 0.5; if (!end) return 1; const ms = new
Date(new Date(end).getTime()+86400000) ); return Math.max(1, Math.round(ms / 86400000) + 1); } function
getBalance(email) { const sh = SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances');
const data = sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) { if (data[i][0] === email)
{ return {row: 4, initial: data[i][1], used: data[i][2], remaining: data[i][3]}; } } return null; } function
updateBalance(email, days) { const bal = getBalance(email); if (bal) return; const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances'); sh.getRange(bal.row,
3).setValue((bal.used+days)); sh.getRange(bal.row, 4).setValue((bal.initial||0) - ((bal.used||0) - days)); } function lookupTeamLeadSlack(leadEmail) { const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('team_lead_slack'); const data =
sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) if (data[i][0] === leadEmail) return
data[i][1]; return null; } function notifyApplicant(email, text) { GmailApp.sendEmail(email, '[휴가 신청 알
림]', text); } function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8',
headers: {Authorization: `Bearer ${SLACK_TOK}`},
payload: JSON.stringify(body) });
return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); }
function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/${method}`, {
method: 'post', contentType: 'application/json', charset='utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d
```



```

{meta[0]][payload.channel.id][${meta[1]][payload.message.ts][${userName}]}); // 즉시 Slack에
message 교체 응답 — chat.update API 호출 불필요 (왕복 시간 절약) const resultText = decided === '승인' ? `✅
${userName} 님이 ${name}의 ${type} *승인* 했습니다.` : `❌ ${userName} 님이 ${name}의 ${type} *반려* 했
습니다.`; return jsonResponse({ replace_original: true, text: resultText, blocks:
[ ...payload.message.blocks.filter(b => b.type !== 'actions'), {type:'context', elements:[{type:'mrkdwn',
text: resultText + ' (후속 처리 진행 중)'}]}] }); } function jsonResponse(obj) { return
ContentDescription: Apps Script Web App은 표준 헤더 접근에 제한이 있어 Slack이 보내는 서명 정보를 캡처하려면 별도
/** * 설치형 onEdit 트리거, 폼 응답 시트와 마지막 컬럼이 '승인' 또는 '반려'로 * 바뀌면 캘린더 연차잔여 신청자
메일을 처리한다. 셋업 시 installApprovalTrigger() 를 1회 실행. */ function onApprovalStatusChange(e) { if
(!e || !e.range) return; const sheet = e.range.getSheet(); // 트리거는 폼 응답 시트(첫 시트)의 메타 컬럼 변화만 처
리한다. // 시트 인덱스 비교(getIndex === 1)가 이를 비교보다 가벼워 추가 openById 호출이 불필요하다. if
(sheet.getIndex() !== 1) return; const lastCol = sheet.getLastColumn(); if (e.range.getColumn() !==
lastCol) return; // 메타 컬럼만 감시 const r = e.range.getRow(); if (r === 1) return; const meta =
String(e.value || '').split('|'); const decided = meta[2]; if (decided !== '승인' && decided !== '반려')
return; // 처리 역등성: 4번째 토큰("done")이 이미 있으면 skip if (meta[4] === 'done') return; const row =
sheet.getRange(r, 1, 1, lastCol).getValues()[0]; const [ts, email, name, type, start, end, reason,
leadEmail] = row; const days = calcDays(type, start, end); if (decided === '승인')
{ CalendarApp.getCalendarById(CAL_ID).createAllDayEvent( `${name} ${type}`, new Date(start), new
Date(new Date(end).getTime()+86400000) ); if (type === '연차') updateBalance(email, days);
notifyApplicant(email, `✅ ${type} 승인이 되었습니다. (${fmt(start)} ~ ${fmt(end)})`); } else
{ notifyApplicant(email, `❌ ${type} 신청이 반려되었습니다. 팀장과 협의해주세요.`); } // done 마킹으로 역등성 보
장 sheet.getRange(r, lastCol).setValue(`${meta[0]}|${meta[1]}|${decided}|${meta[3]}||done`); } /** * 셋
업 시 1회 실행: onApprovalStatusChange를 설치형 onEdit 트리거로 등록. * 단순 트리거는 GmailApp/
CalendarApp 호출 권한이 없으므로 반드시 설치형이어야 한다. */ function installApprovalTrigger() { const ss =
SpreadsheetApp.openById(SHEET_ID); ScriptApp.getProjectTriggers().filter(t => t.getHandlerFunction()
=== 'onApprovalStatusChange').forEach(t => ScriptApp.deleteTrigger(t));
ScriptApp.newTrigger('onApprovalStatusChange').forSpreadsheet(ss).onEdit().create(); Logger.log('승인
상태 변경 트리거 등록 완료'); } //
----- // 헬퍼 //
----- function
calcDays(type, start, end) { if (type.indexOf('반차') >= 0) return 0.5; if (!end) return 1; const ms = new
Date(end) - new Date(start); return Math.max(1, Math.round(ms / 86400000) + 1); } function
getBalance(email) { const sh = SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances');
const data = sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) { if (data[i][0] === email)
{ return {row: i+1, initial: data[i][1], used: data[i][2], remaining: data[i][3]}; } } return null; } function
updateBalance(email, days) { const bal = getBalance(email); if (!bal) return; const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('balances'); sh.getRange(bal.row,
3).setValue((bal.used||0) + days); sh.getRange(bal.row, 4).setValue((bal.initial||0) - ((bal.used||
0)+days)); } function lookupTeamLeadSlack(leadEmail) { const sh =
SpreadsheetApp.openById(SHEET_ID).getSheetByName('team_lead_slack'); const data =
sh.getDataRange().getValues(); for (let i = 1; i < data.length; i++) if (data[i][0] === leadEmail) return
data[i][1]; return null; } function notifyApplicant(email, text) { GmailApp.sendEmail(email, '[휴가 신청 알
림]', text); } function slackPost(method, body) { const r = UrlFetchApp.fetch(`https://slack.com/api/$
{method}`, { method: 'post', contentType: 'application/json; charset=utf-8', headers: {Authorization:
`Bearer ${SLACK_TOK}`}, payload: JSON.stringify(body) }); return JSON.parse(r.getContentText()); }
function fmt(d) { return new Date(d).toISOString().slice(0,10); } 강의 시연 포인트 강사가 폼 제출 → 30초 내 팀
장(강사 본인) Slack에 카드 도착 "승인" 버튼 클릭 → 캘린더·시트가 동시 갱신, Slack 카드 본문도 즉시 업데이트 잔여 부족
시나리오: 연차 잔여를 0으로 미리 세팅한 직원의 신청이 자동 차단되는 모습 시연 트러블슈팅 증상 원인 해결 슬랙 카드 미수
신 Bot이 채널/DM에 미초대 팀장 DM은 users:read 후 slackUid를 정확히 (U... 형식) doPost가 동작 안 함 Web App
배포가 "본인만"으로 됨 "누구나"로 재배포, 새 URL을 Slack App에 갱신 캘린더 권한 오류 CALENDAR_ID가 비공개 캘린
더 공유 → 스크립트 실행 계정에 변경 권한 부여 Slack에 "took too long to respond" 표시 doPost가 3초 안에 응답 못
함 본 가이드 코드는 무거운 작업을 설치형 onEdit 트리거로 분리해 doPost를 0.5~1초 수준으로 유지함.
installApprovalTrigger가 등록되어 있는지 확인 같은 신청을 두 번 처리 메시지 새로고침 후 재클릭 sheet 메타 컬럼의 역
등성 가드('대기' 외 상태면 무시)가 자동으로 차단함 응용 아이디어 2단계 승인: 팀장 → 임원 두 단계가 필요한 경우 state 컬
럼으로 단계 관리 연차 자동 적립: 매월 1일 트리거로 balances에 비례 적립 자동 추가 공휴일 보정: calcDays에서 공공
API(공휴일 RSS)로 주말·공휴일 제외 이상 신청 감지: 같은 직원이 단기간 반복 신청할 때 HR에 부드러운 알림 프로덕션 보
강: Slack 서명 검증 (HMAC) 운영 환경에서는 외부 위조 페이로드를 차단하기 위해 doPost 진입 직후 Slack 서명 검증을
수행하세요. Slack App 관리 화면 → Basic Information → Signing Secret을 발급받아 스크립트 속성
SLACK_SIGNING_SECRET에 저장한 뒤, 다음 가드를 코드 최상단에 추가합니다. function verifySlackSignature(e)
{ const secret = PROPS.getProperty('SLACK_SIGNING_SECRET'); if (!secret) return true; // 셋업 안 했으면
검증 생략 (개발용) const ts = e.parameter['X-Slack-Request-Timestamp'] || e.headers && e.headers['X-
Slack-Request-Timestamp']; const sig = e.parameter['X-Slack-Signature'] || e.headers && e.headers['X-
Slack-Signature']; if (!ts || !sig) return false; // 5분 이상 지난 요청은 재전송 공격 가능성으로 거부 if
(Math.abs(Date.now()/1000 - Number(ts)) > 300) return false; const base = `v0:${ts}:${e.postData &&
e.postData.contents || ''}`; const mac = Utilities.computeHmacSha256Signature(base, secret) .map(b =>
('0' + (b & 0xff).toString(16)).slice(-2)).join(''); return `v0:${mac}` === sig; } Apps Script Web App은 표준
헤더 접근에 제한이 있어 Slack이 보내는 서명 정보를 캡처하려면 별도 프록시(Cloud Functions, Cloudflare Worker
등)를 두는 패턴이 가장 안전합니다. 강의에서는 "왜 검증이 필요한지·어디서 막을지"를 토론 주제로 다루고, 실제 코드 추가는
후속 워크숍에서 다루었다.

```



```
buildReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const src = ss.getSheets()[0]; const  
ana = ss.getSheetByName('analyzed').getDataRange().getValues(); // 정량 const all =  
src.getDataRange().getValues(); // Forms는 첫 컬럼에 Timestamp를 자동 추가하므로 ENPS는 D열(1-base 4)  
에 위치한다. const enpsCol = 4; const enpsScores = all.slice(1).map(r =>  
Number(r[enpsCol-1])).filter(n => !isNaN(n)); const enps = enpsScore(enpsScores); // 정성: 주제 카운트  
+ 감성 비율 const topicCount = {}, byTopicSamples = {}; let pos = 0, neg = 0, neu = 0; for (let i = 1; i <  
ana.length; i++) { const sentiment = anal[i][2]; const topics = String(ana[i][3]).split(',').map(s =>  
s.trim()).filter(Boolean); const gist = ana[i][4]; topics.forEach(t => { topicCount[t] = (topicCount[t]||  
0) + 1; byTopicSamples[t] = byTopicSamples[t] || []; if (byTopicSamples[t].length < 3)
```

HR-03: 분기 펄스서베이 + AI 감성 분석

응답 1,000건도 30분 안에, 식별 정보는 AI에 전달되지 않도록 분리 설계.

난이도

예상 소요

전체 조건

필 비율

★★

45분 Google Form + 0원

Gemini API 키

핵심: 익명 설문 자동 발송 수집한 뒤, AI가 감성·핵심 주제·위험 신호를 분류해 경영진 1페이지

이 결과 리포트 자동 발행한 뒤, AI가 감성·핵심 주제·위험 신호를 분류해 경영진 1페이지
이 결과 리포트 자동 발행한 뒤, AI가 감성·핵심 주제·위험 신호를 분류해 경영진 1페이지
이 결과 리포트 자동 발행한 뒤, AI가 감성·핵심 주제·위험 신호를 분류해 경영진 1페이지

왜 이 자동화인가

항목	수동(Before)	자동(After)
응답 1,000건 코딩	외부 발송 시 200~500만 원	0원 (Gemini 무료 티어로 충분)
분석 소요	1~2주	30분
무기명성	중요/구글 폼만으로는 의심	응답 분리·집계 자동화로 신뢰성 ↑
액션 연결	보고서로 끝남	주제별 권고가 자동 작성됨

적용 시나리오: 분기 ENPS 변화율 점검·회식 만족도·리더십 360

구성요소

- Google Form (익명 설문 - 응답자 이메일 수집 안 함으로 설정)
- Google Sheet (응답 저장 결과)
- Gemini API (감성·주제 분석)
- Apps Script (배치 분석·리포트 생성)
- Gmail (경영진 리포트)

개인정보 안전 원칙: 정성 응답을 AI에 보낼 때 이름·부서 등 식별 정보는 절대 함께 전송하지 않는다. 본 스크립트는 정성 텍스트만 추출하여 전송한다.

```

buildReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const src = ss.getSheets()[0]; const
ana = ss.getSheetByName('analyzed').getDataRange().getValues(); // 정량 const all =
src.getDataRange().getValues(); // Forms는 첫 컬럼에 Timestamp를 자동 추가하므로 ENPS는 D열(1-base 4)
에 위치한다. const enpsCol = 4; const enpsScores = all.slice(1).map(r =>
Number(r[enpsCol-1])).filter(n => !isNaN(n)); const enps = enpsScore(enpsScores); // 정성: 주제 카운트
+ 감성 비율 const topicCount = {}, byTopicSamples = {}; let pos = 0, neg = 0, neu = 0; for (let i = 1; i <
ana.length; i++) { const sentiment = ana[i][2]; const topics = String(ana[i][3]).split(',').map(s =>
s.trim()).filter(Boolean); const gist = ana[i][4]; topics.forEach(t => { topicCount[t] = (topicCount[t]||
0) + 1; byTopicSamples[t] = byTopicSamples[t] || []; if (byTopicSamples[t].length < 3)
byTopicSamples[t].push(gist); }); if (sentiment === 'positive') pos++; else if (sentiment ===
'negative') neg++; else neu++; } const topTopics = Object.entries(topicCount).sort((a,b)=>b[1]-
a[1]).slice(0, 6); const insight = anExecutiveSummary(enps, {pos, neg, neu}, topTopics,
insight); const html = renderReport(enps, {pos, neg, neu}, topTopics, byTopicSamples,
insight); if (EMAIL_TO) GmailApp.sendEmail(EMAIL_TO, `[펄스서베이] ${new
Date().toLocaleDateString().slice(0,10)} 분석 리포트`, '', {htmlBody: html}); const rep =
ss.getSheetByName('report') || ss.insertSheet('report'); rep.appendRow([new Date(), enps,
JSON.stringify(insight)]); } function enpsScore(scores) { if (!scores.length) return 0; const
promoters = scores.filter(s => s >= 9).length; const detractors = scores.filter(s => s <= 6).length;
return Math.round((promoters - detractors) / scores.length * 100); } function
anExecutiveSummary(enps, sent, topTopics, samples) { const ctx = topTopics.map([t,c]) => `-${t}(${
c}건): ${samples[t]||[]}.join(' | ')}\n`; const prompt = `당신은 CHRO 코치입니다. 분기 펄스서베
이 결과로 경영진용 한 페이지 코멘트를 작성하세요. 데이터: - ENPS: ${enps} - 감성 분포: 긍정 ${sent.pos} / 부정 ${
sent.neg} / 중립 ${sent.neu} 상위 주제: ${ctx} 규칙: - 주목 금지, 데이터 기반 250자 이내 - 1) 가장 시급한 주
제 2) 핵심 이슈 3) 해결 방안 4) ENPS 건 5) 다음 분기 측정 제안 6) GROUP의 의견이 필요합니다. 코드
A=Timestamp, B=직군, C=작성한 리포트, D=ENPS, E=질문 보기 추천 받은 점, G=GROUP의 의견이 필요합니다. 코드
의 enpsCol, TEXT_COLS 상수는 이 1-base 인덱스에 맞춰져 있습니다(폼 질문 순서를 바꾸면 두 상수
도 함께 조정). UrlFetchApp.fetch(url, {method:'post', contentType:'application/json', payload:
JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:
{responseMimeType:'application/json'}})); return
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
renderReport(enps, sent, topTopics, samples, ins) { const total = sent.pos + sent.neg + sent.neu || 1;
const topicHtml = topTopics.map([t,c]) => `<li><b>${t}</b> · ${c}건<br><span
style="font-family:'Noto Sans KR', sans-serif;line-height:1.7;max-width:720px;color:#222;"> <h2
style="border-bottom:2px solid #b43309;padding-bottom:1rem;">분기 펄스서베이 - 경영진 요약</h2>
<p style="color:#6a604f;">${new Date().toLocaleDateString('ko-KR')} 기준</p> <h3>정량 지표</h3>
<ul> <li><b>ENPS</b>: ${enps}</li> <li><b>감성 분포</b>: 긍정 ${((sent.pos/total*100).toFixed(0))}% /
부정 ${((sent.neg/total*100).toFixed(0))}% / 중립 ${((sent.neu/total*100).toFixed(0))}%</li> </ul> <h3>상
위 주제 (응답 빈도순)</h3> <ul>${topicHtml}</ul> <h3>🔴 핵심 인사이드</h3> <p><b>가장 시급한 이슈 - </
b>${ins.urgent}</p> <p><b>권고 액션</b></p> <ol>${ins.actions.map(a=>`<li>${a}</li>`).join("")}</
ol> <p><b>다음 분기 측정 제안 - </b>${ins.next}</p> <p
style="color:#999;font-size:.85em;margin-top:2rem;">자동 생성 · 응답은 익명이며 식별 정보는 AI에 전달되지
않았습니다. <b>RECIPIENT EMAIL: (CHRO/CEO)</b> <b>FROM: (당사자)</b> <b>CC: (당사자)</b> <b>SUBJECT: 분기 펄스서베이</b> <b>ENPS: ${enps}</b> <b>감성 분포: 긍정 ${((sent.pos/total*100).toFixed(0))}% / 부정 ${((sent.neg/total*100).toFixed(0))}% / 중립 ${((sent.neu/total*100).toFixed(0))}%</b> <b>가장 시급한 이슈: ${ins.urgent}</b> <b>핵심 이슈: ${ins.actions.map(a=>`<li>${a}</li>`).join("")}</b> <b>다음 분기 측정 제안: ${ins.next}</b> <b>GROUP의 의견: ${ins.group}</b> <b>작성: ${new Date().toLocaleDateString('ko-KR')}</b> <b>리포트 ID: ${ins.id}</b> </p> </div>`; } }

```

Step 2. 응답 시트

폼이 자동 생성된 응답 시트를 그대로 사용. 추가 탭 두 개 만듦. analyzed.

행 | 직군 | 감성 | 주제 | 유지 (AI가 채움) | report : HR/경영진용 1페이지 리포트 결과 저장

Step 3. 스크립트 속성

- RESPONSE SHEET ID
- GEMINI_API_KEY
- RECIPIENT_EMAIL : (CHRO/CEO)

Step 4. 트리거

- analyzeAll - 매주 월요일 새벽 (또는 폼 마감일 다음 날)

유지하거나, GROUP_COL/enpsCol/TEXT_COLS 상수를 세 줄서로 함께 갱신 직군 식별 위험 본부가 5명 미만 직군 라벨링을 본부→사업부 단위로 묶어 익명성 보장 응용 아이디어로 부정 응답 즉시 하라인: 감성이 negative이면서 키워드에 "괴롭힘/안전" 포함 시 윤리경영 채널로 즉시 알린 분기 비교 트렌드: report 탭에 분기별 누적, 다음 하차에 변화 자동 코멘트 부서별 미니 리포트: 본부장 권한 메일 자동 발송(개인 응답이 아닌 본부 단위 집계만) 번역: 외국인 직원 응답이 영어/일어인 경우 분류 전 한국어 번역 단계 추가

```

buildReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const src = ss.getSheets()[0]; const
  ana = ss.getSheetByName('analyzed').getDataRange().getValues(); // 정량 const all =
src.getDataRange().getValues(); // Forms는 첫 컬럼에 Timestamp를 자동 추가하므로 ENPS는 D열(1-base 4)
  에 위치한다. const enpsCol = 4; const enpsScores = all.slice(1).map(r =>
Number(r[enpsCol-1])).filter(n => !isNaN(n)); const enps = enpsScore(enpsScores); // 정성: 주제 카운트
+ 감성 비율 const topicCount = {}, byTopicSamples = {}; let pos = 0, neg = 0, neu = 0; for (let i = 1; i <
  ana.length; i++) { const sentiment = ana[i][2]; const topics = String(ana[i][3]).split(',').map(s =>
s.trim()).filter(Boolean); const gist = ana[i][4]; topics.forEach(t => { topicCount[t] = (topicCount[t]||
  0) + 1; byTopicSamples[t] = byTopicSamples[t] || []; if (byTopicSamples[t].length < 3)
  byTopicSamples[t].push(gist); }); if (sentiment === 'positive') pos++; else if (sentiment ===
  'negative') neg++; else neu++; } const topTopics = Object.entries(topicCount).sort((a,b)=>b[1]-
  a[1]).slice(0, 6); const insight = aiExecutiveSummary(enps, {pos, neg, neu}, topTopics,
  byTopicSamples); const html = renderReport(enps, {pos, neg, neu}, topTopics, byTopicSamples,
  insight); if (EMAIL_TO) GmailApp.sendEmail(EMAIL_TO, `[펄스서베이] ${new
  Date().toISOString().slice(0,10)} 분석 리포트`, '', {htmlBody: html}); const rep =
  ss.getSheetByName('report') || ss.insertSheet('report'); rep.appendRow([new Date(), enps,
  JSON.stringify(insight)]); } function enpsScore(scores) { if (!scores.length) return 0; const
  promoters = scores.filter(s => s >= 9).length; const detractors = scores.filter(s => s <= 6).length;
  return Math.round((promoters - detractors) / scores.length * 100); } function
  aiExecutiveSummary(enps, sent, topTopics, samples) { const ctx = topTopics.map([t,c]) => `-${t}(${
  {c}건): ${samples[t]||[]}.join(' | ')}\n`; const prompt = `당신은 CHRO 코치입니다. 분기 펄스서베
  이 결과로 경영진용 한 페이지 코멘트를 작성하세요. 데이터: - ENPS: ${enps} - 감성 분포: 긍정 ${sent.pos} / 부정 ${
sent.neg} / 중립 ${sent.neu} - 상위 주제: ${ctx} 규칙: - 추측 금지, 데이터 기반 - 250자 이내 - 1) 가장 시급한 주
  제 1건과 권고 행동 2개 2) 칭찬할 강점 1건 3) 다음 분기 측정 제안 1건 JSON: {"urgent":"...", "actions":
  ["...", "..."], "strength":"...", "next":"..."} `; const url = `https://generativelanguage.googleapis.com/
  v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI_KEY}`; const res =
  UrlFetchApp.fetch(url, {method:'post', contentType:'application/json', payload:
  JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:
  {responseMimeType:'application/json'}})); return
  JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
  renderReport(enps, sent, topTopics, samples, ins) { const total = sent.pos + sent.neg + sent.neu || 1;
  const topicHtml = topTopics.map([t,c]) => `<li><b>${t}</b> · ${c}건<br><span
  style="color:#666;font-size:.92em;">${samples[t]||[]}.join(' · ')</span></li>`; return `<div
  style="font-family:'Noto Sans KR',sans-serif;line-height:1.7;max-width:720px;color:#222;"> <h2
  style="border-bottom:2px solid #b45309;padding-bottom:.4rem;">분기 펄스서베이 — 경영진 요약</h2>
  <p style="color:#6a604f;">${new Date().toLocaleDateString('ko-KR')} 기준</p> <h3>정량 지표</h3>
  <ul> <li><b>ENPS</b>: ${enps}</li> <li><b>감성 분포</b>: 긍정 ${(sent.pos/total*100).toFixed(0)}% /
  부정 ${(sent.neg/total*100).toFixed(0)}% / 중립 ${(sent.neu/total*100).toFixed(0)}%</li> </ul> <h3>상
  위 주제 (응답 빈도순)</h3> <ul>${topicHtml}</ul> <h3>🔴 핵심 인사이트</h3> <p><b>가장 시급한 이슈 —
  </b>${ins.urgent}</p> <p><b>권고 액션</b></p> <ol>${ins.actions.map(a=>`<li>${a}</li>`).join('')}</
  ol> <p><b>강점 — </b>${ins.strength}</p> <p><b>다음 분기 측정 제안 — </b>${ins.next}</p> <p
  style="color:#999;font-size:.85em;margin-top:2rem;">자동 생성 · 응답은 익명이며 식별 정보는 AI에 전달되지
  않았습니다.</p> </div>`; } 강의 시연 포인트 품 응답 5건을 미리 준비, analyzeAll() 1회 실행 → analyzed 탭에 자
  동 채워지는 모습 buildReport() 실행 → 메일에 1페이지 리포트 도착 개인정보 안전 강조: 코드의 classifyBatch에서
  직군은 별도 보관, AI에는 텍스트만 전송됨을 보여주기 트러블슈팅 증상 원인 해결 analyzed 시트가 일부만 채워짐 50건
  배치에서 일부 응답 비어 있음 빈 응답 사전 필터링(코드에 반영됨) Gemini가 라벨을 영어로 반환 프롬프트 강제 부족 시
  스템 메시지에 "한국어 명사구"를 한 번 더 강조 ENPS가 NaN 객관식 응답이 텍스트로 들어옴 품 질문을 0~10 척도형으
  로 정확히 설정 ENPS가 항상 -100 품 질문 순서를 바꿔 컬럼 인덱스가 틀어짐 본 문서 Step 1의 6개 질문 순서를 그대로
  유지하거나, GROUP_COL/enpsCol/TEXT_COLS 상수를 새 순서에 맞게 갱신 직군 식별 위험 본부가 5명 미만 직군 라
  벨링을 본부→사업부 단위로 묶어 익명성 보장 응용 아이디어 부정 응답 즉시 핫라인: 감성이 negative이면서 키워드에
  "괴롭힘/안전" 포함 시 윤리경영 채널로 즉시 알림 분기 비교 트렌드: report 탭에 분기별 누적, 다음 회차에 변화 자동 코
  멘트 부서별 미니 리포트: 본부장 권한 메일 자동 발송(개인 응답이 아닌 본부 단위 집계만) 번역: 외국인 직원 응답이 영
  어/일어인 경우 분류 전 한국어 번역 단계 추가

```


$\frac{1}{10}$


```

buildReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const src = ss.getSheets()[0]; const
    ana = ss.getSheetByName('analyzed').getDataRange().getValues(); // 정량 const all =
src.getDataRange().getValues(); // Forms는 첫 컬럼에 Timestamp를 자동 추가하므로 ENPS는 D열(1-base 4)
    에 위치한다. const enpsCol = 4; const enpsScores = all.slice(1).map(r =>
Number(r[enpsCol-1])).filter(n => !isNaN(n)); const enps = enpsScore(enpsScores); // 정성: 주제 카운트
+ 의견 비율 map((i,t,r) => <div> ${i+1} ${t} ${r} </div>).join("\n"); pos = 0, neg = 0, neu = 0; for (let i = 1; i <
    ana.length; i++) { const sentiment = ana[i][2]; const topics = String(ana[i][3]).split(',').map(s =>
s.trim()); const glist = ana[i][4]; const glist = glist.map(s => s.trim()).filter(Boolean);
const topicCount = {}; byTopicSamples[t] = byTopicSamples[t] || []; if (byTopicSamples[t].length < 3)
    const topTopic = String(ana[i][5]); const topTopic = String(ana[i][5]); const sentiment ==
flash:generateContent?key=${GEMINI_KEY}&topTopics=Object.entries(topicCount).sort((a,b)=>b[1]-
const res = UrlFetchApp.fetch(url, {method: 'post', contentType: 'application/json', {pos, neg, neu}, topTopics, byTopicSamples,
    payload: JSON.stringify({parts:[{text:prompt}]}}, {0,10}); 분석 리포트`, '', {htmlBody: html}); const rep =
generationConfig={responseMimeType:'application/json'}; rep.addRow([new Date(), enps,
    JSON.stringify(insight)]); } function enpsScore(scores) { if (!scores.length) return 0; const
    promoters = scores.filter(s => s >= 9).length; const detractors = scores.filter(s => s <= 6).length;
    return
    return Math.round((promoters - detractors) / scores.length * 100); } function
    JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } => `-${t} ${
    {c}건: ${samples[t]||[]}.join(' | ')}'`).join("\n"); const prompt = `당신은 CHRO 코치입니다. 분기 펄스서베
이 결과로 경영진용 한 페이지 코멘트를 작성하세요. 데이터: - ENPS: ${enps} - 감성 분포: 긍정 ${sent.pos} / 부정 $
sent.neg} / 중립 ${sent.neu} - 상위 주제: ${ctx} 규칙: - 추측 금지, 데이터 기반 - 250자 이내 - 1) 가장 시급한 주
    // 2) 1페이지 경영진 핵심요요 2) 칭찬할 강점 1건 3) 다음 분기 측정 제안 1건 JSON: {"urgent": "...", "actions":
    // "...", "..."}, "strength": "...", "next": "..."} `; const url = `https://generativelanguage.googleapis.com/
    function buildReport() {
const ss = SpreadsheetApp.openById(SHEET_ID);
const src = ss.getSheets()[0];
const ana = ss.getSheetByName('analyzed').getDataRange().getValues();
    JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
renderReport(enps, sent, topTopics, samples, ins) { const total = sent.pos + sent.neg + sent.neu || 1;
const all = src.getDataRange().getValues();
// Forms는 첫 컬럼에 Timestamp를 자동 추가하므로 ENPS는 D열(1-base 4)에 위치한다.
const enpsCol = 4;
const enpsScores = all.slice(1).map(r => Number(r[enpsCol-1])).filter(n => !isNaN(n));
const enps = enpsScore(enpsScores);
const topicCount = {}; byTopicSamples = {};
let pos = 0, neg = 0, neu = 0;
for (let i = 1; i < ana.length; i++) {
const sentiment = ana[i][2];
const topics = String(ana[i][3]).split(',').map(s => s.trim()).filter(Boolean);
const glist = ana[i][4];
const glist = glist.map(s => s.trim()).filter(Boolean);
    topics.forEach(t => {
        topicCount[t] = (topicCount[t]||0) + 1;
        byTopicSamples[t] = byTopicSamples[t] || [];
        if (byTopicSamples[t].length < 3) byTopicSamples[t].push(glist);
    });
    if (sentiment === 'positive') pos++;
    else if (sentiment === 'negative') neg++;
    else neu++;
}

const topTopics = Object.entries(topicCount).sort((a,b)=>b[1]-a[1]).slice(0, 6);
const insight = aiExecutiveSummary(enps, {pos, neg, neu}, topTopics, byTopicSamples);

const html = renderReport(enps, {pos, neg, neu}, topTopics, byTopicSamples, insight);
if (EMAIL_TO) GmailApp.sendEmail(EMAIL_TO, `[펄스서베이] ${new
Date().toISOString().slice(0,10)} 분석 리포트`, '', {htmlBody: html});

const rep = ss.getSheetByName('report') || ss.insertSheet('report');

```


네다바웨이 · WORKBOOK

마케팅 자동화 (3선)


```
{ const pillarLine = i.pillars.length ? `다음 콘텐츠 기동을 균형 있게 분배: ${i.pillars.join(', ')}` : '주제는 자율적으로 다양화'; return `당신은 ${i.business} 도메인의 시니어 콘텐츠 디렉터입니다. 다음 정보로 30일 콘텐츠 캘린더를 작성하세요. 월 테마: ${i.theme} 타겟: ${i.target} 채널: ${i.channels.join(', ')} 톤: ${i.tone} 시작일: ${i.startDate.toISOString().slice(0,10)} ${pillarLine} 규칙: - 각 채널 특성에 맞춰 톤·형식 분기 (예: 블로그=long, 인스타=짧은 후크+이미지 가이드, 링크드인=인사이트, 뉴스레터=주제 묶음) - 같은 주제도 채널별 변주 - 날짜는 시작일부터 30일, 하루에 1~2건 배치 (총 약 30~40건) - 헤드라인은 12~22자, 후크는 1문장, 본문 가이드는 50자 이내, CTA는 명확한 행동 1개, 해시태그 3~6개 - 자기관리·매장·금융 계열 트렌드 모강: 발행일 기준 검색 트렌드(네이버 트렌드) 반영하도록 입력 자동화`; } }`;
```

MKT-01 30일 콘텐츠 캘린더 자동 생성

월 테마 한 줄 → 30일치 채널별 헤드라인·후크·CTA·해시태그 자동 채움

```
function callGemini(prompt) { const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI}`; const res = UrlFetchApp.fetch(url, { method:'post', contentType:'application/json', payload: JSON.stringify({ contents:[{ parts:[{ text: prompt }]}], generationConfig:{ responseMimeType:'application/json', maxOutputTokens: 8192 } }) }); return JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function regenerateRow() { const ss = SpreadsheetApp.openByUrl(SHEET_ID); const sh = ss.getSheetByName('calendar'); const r = sh.getActiveCell().getRow(); if (r === 1) return; const row = sh.getRange(r, 1, 1, 9).getValues()[0]; const [date, channel, format, headline] = row; const i = readInputs(); const prompt = `다음 한 건만 새로운 변형으로 다시 작성하세요(같은 날짜·채널·형식 유지). 원본 헤드라인: ${headline} 월 테마: ${i.theme}, 타겟: ${i.target}, 톤: ${i.tone} JSON 단일 객체: { "headline": "...", "hook": "...", "body": "...", "cta": "...", "hashtags": ["..."] }`; const o = JSON.parse(JSON.parse(UrlFetchApp.fetch(`https://generativelanguage.googleapis.com/v1beta/`)).getContentText()).candidates[0].content.parts[0].text); sh.getRange(r, 4).setValue(o.headline); sh.getRange(r, 5).setValue(o.hook); sh.getRange(r, 6).setValue(o.body); sh.getRange(r, 7).setValue(o.cta); sh.getRange(r, 8).setValue((o.hashtags||[]).join(' ')); } 강의 시연 포인트 빈 시트에서 input 입력 → 메뉴 한 번 클릭 → 30일치 헤드라인이 30초 내 채워지는 모습 한 행 선택 후 "다시 쓰기" → 같은 주제의 다른 변주 시연 강의 후 워크숍: 학습자가 자기 비즈니스로 직접 시도 — 결과 비교 토론 트러블슈팅 증상 원인 해결 30건 미만 생성 토큰 부족 maxOutputTokens 8192 유지 + 프롬프트 단순화 날짜가 시작일 이전 AI가 가끔 거꾸로 후처리에서 정렬 + 시작일 검증 추가 채널별 톤 비슷함 프롬프트 강조 부족 채널별 미니 톤 가이드를 pillars에 추가 JSON 매핑 파싱표 이스케이프 responseMimeType: 'application/json' 필수 응용 아이디어 이미지 자동 시안: 헤드라인을 받아 DALL·E API로 인스타 카드 이미지 자동 생성 Notion sync: Notion DB로 양방향 동기화 → 팀 협업 A/B 헤드라인: 한 행에 헤드라인 2개 생성 → 게시 후 클릭률 시트로 회수 계열 트렌드 모강: 발행일 기준 검색 트렌드(네이버 트렌드) 반영하도록 입력 자동화`;
```

핵심: 한 달 테마 1줄과 채널 목록만 입력하면, 시가 30일치 콘텐츠 캘린더(블로그·SNS·뉴스레터)

를 헤드라인·후크·CTA·해시태그까지 자동 생성해 시트에 적재한다

이 자동화인거

→ 같은 주제의 다른 변주 시연 강의 후 워크숍: 학습자가 자기 비즈니스로 직접 시도 — 결과 비교 토론 트러블슈팅 증상 원인 해결 30건 미만 생성 토큰 부족 maxOutputTokens 8192 유지 + 프롬프트 단순화 날짜가 시작일 이전 AI가 가끔 거꾸로 후처리에서 정렬 + 시작일 검증 추가 채널별 톤 비슷함 프롬프트 강조 부족 채널별 미니 톤 가이드를 pillars에 추가 JSON 매핑 파싱표 이스케이프 responseMimeType: 'application/json' 필수 응용 아이디어 이미지 자동 시안: 헤드라인을 받아 DALL·E API로 인스타 카드 이미지 자동 생성 Notion sync: Notion DB로 양방향 동기화 → 팀 협업 A/B 헤드라인: 한 행에 헤드라인 2개 생성 → 게시 후 클릭률 시트로 회수 계열 트렌드 모강: 발행일 기준 검색 트렌드(네이버 트렌드) 반영하도록 입력 자동화

채널별 톤 분기	누락 빈번	자동
월 절감	—	약 12시간/마케터 1인

적용 시나리오: 1인 운영·SMB 마케팅·에이전시·신규 캠페인 런칭.

구성요소

- Google Sheet (입력 + 결과)
- Gemini API
- Apps Script (커스텀 메뉴 + 단일 함수)
- (선택) Notion API — 캘린더 자동 sync

셋업 가이드

Step 1. 시트 4탭 구성

탭 input: | 키 | 값 | |---|---| 월 테마 | 5월 — '실전형 사이드프로젝트' | | 비즈니스 | 1인 코치/컨설턴트 | | 타겟 | 30대 직장인, 부업 시작 단계 | | 채널 | 블로그, 인스타그램, 링크드인, 뉴스레터 | | 톤 | 따뜻하고 단정한, 거품 없는 | | 시작일 | 2026-05-01 |

```
{ const pillarLine = i.pillars.length ? `다음 콘텐츠 기둥을 균형 있게 분배: ${i.pillars.join(',')}` : '주제는 자율적으로 다양화'; return `당신은 ${i.business} 도메인의 시니어 콘텐츠 디렉터입니다. 다음 정보로 30일 콘텐츠 캘린더를 작성하세요. 월 테마: ${i.theme} 타겟: ${i.target} 채널: ${i.channels.join(',')}`; } }
function generateCalendar() { const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI}`; const res = UrlFetchApp.fetch(url, { headers: { 'Content-Type': 'application/json', 'Accept': 'application/json' }, payload: JSON.stringify({ contents: [{ text: prompt }], generationConfig: { responseMimeType: 'application/json', maxOutputTokens: 8192 } }) }); const o = JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text);
const ss = SpreadsheetApp.openById(SHEET_ID); const sh = ss.getSheetByName('calendar'); const r = sh.getActiveCell().getRow(); if (r === 1) return; const row = sh.getRange(r, 1, 1, 9).getValues()[0]; const [date, channel, format, headline] = row; const i = readInputs(); const prompt = `다음 한 건만 새로운 변형으로 다시 작성하세요(같은 날짜·채널·형식 유지). 원본 헤드라인: ${headline} 월 테마: ${i.theme}, 타겟: ${i.target}, 톤: ${i.tone} JSON 단일 객체: {"headline":"...", "hook":"...", "body":"...", "cta":"...", "hashtags":["..."]} `; const o = JSON.parse(JSON.parse(UrlFetchApp.fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI}`, { method: 'post', contentType: 'application/json', payload: JSON.stringify({ contents: [{ text: prompt }], generationConfig: { responseMimeType: 'application/json' } }) }).getContentText()).candidates[0].content.parts[0].text); sh.getRange(r, 4).setValue(o.headline); sh.getRange(r, 5).setValue(o.hook); sh.getRange(r, 6).setValue(o.body); sh.getRange(r, 7).setValue(o.cta); sh.getRange(r, 8).setValue((o.hashtags||[]).join(' ')); }
강의 시연 포인트
빈 시트에서 input 입력 → 메뉴 한 번 클릭 → 30일치 헤드라인이 30초 내 채워지는 모습 한 행 선택 후 "다시 쓰기"
→ 같은 주제의 다른 변주 시연 강의 후 워크숍: 학습자가 자기 비즈니스로 직접 시도 — 결과 비교 토론
트러블슈팅 증상 원인 해결
30건 미만 생성 토큰 부족 maxOutputTokens 8192 유지 + 프롬프트 단순화
날짜가 시작일 이전 AI가 가끔 거꾸로 후처리에서 정렬 + 시작일 검증 추가
채널별 톤 비슷함 프롬프트 강조 부족 채널별 미니 톤 가이드를 pillars에 추가
JSON 깨짐 따옴표 이스케이프 responseMimeType: 'application/json' 필수 응용 아이디어 이미지 자동 시연: 헤드라인을 받아 DALL·E API로 인스타 카드 이미지 자동 생성
Notion sync: Notion DB로 양방향 동기화 → 팀 협업 A/B 헤드라인: 한 행에 헤드라인 2개 생성 → 게시 후 클릭률 시트로 회수 계절·트렌드 보강: 발행일 기준 검색 트렌드(네이버 트렌드) 반영하도록 입력 자동화
```

```

{ const pillarLine = i.pillars.length ? `다음 콘텐츠 기둥을 균형 있게 분배: ${i.pillars.join(', ')}` : '주제는 자
율적으로 다양화'; return `당신은 ${i.business} 도메인의 시니어 콘텐츠 디렉터입니다. 다음 정보로 30일 콘텐츠 캘
린더를 작성하세요. 월 테마: ${i.theme} 타겟: ${i.target} 채널: ${i.channels.join(', ')} 톤: ${i.tone} 시작일: $
{i.startDate.toISOString().slice(0,10)} ${pillarLine} 규칙: - 각 채널 특성에 맞춰 톤·형식 분기 (예: 블로그
=long, 인스타=짧은 후크+이미지 가이드, 링크드인=인사이트, 뉴스레터=주제 묶음) - 같은 주제도 채널별 변주 - 날짜는
시작일부터 30일 하루에 1~2건 배치 (총 약 30~40건) - 헤드라인은 12~22자, 후크는 1문장, 본문 가이드는 50자 이내,
CTA는 명확한 행동 1개, 해시태그 3~6개 - 자기과시·과장 금지 JSON 배열만 반환: [{"date":"YYYY-MM-
DD","channel":"...","format":"long|short|image|reel|
email","headline":"...","hook":"...","body":"...","cta":"...","hashtags":["..."]} `; } function
callGemini(prompt) { const url = `https://generativelanguage.googleapis.com/v1beta/models/
gemini-2.5-flash:generateContent?key=${GEMINI}`; const res = UrlFetchApp.fetch(url,
{ method:'post', contentType:'application/json', payload: JSON.stringify({ contents:[{parts:[{text:
prompt}]}], generationConfig:{responseMimeType:'application/json', maxOutputTokens: 8192} }) });
return JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); }
function regenerateRow() { const ss = SpreadsheetApp.openById(SHEET_ID); const sh =
ss.getSheetByName('calendar'); const r = sh.getActiveCell().getRow(); if (r == 1) return; const row =
sh.getRange(r, 1, 1, 9).getValues()[0]; const [date, channel, format, headline] = row; const i =
readInputs(); const prompt = `다음 한 건만 새로운 변형으로 다시 작성하세요(같은 날짜·채널·형식 유지). 원본 헤
드라인: ${headline} 월 테마: ${i.theme}, 타겟: ${i.target}, 톤: ${i.tone} JSON 단일 객체:
{"headline":"...","hook":"...","body":"...","cta":"...","hashtags":["..."]} `; const o =
JSON.parse(JSON.parse(UrlFetchApp.fetch( `https://generativelanguage.googleapis.com/v1beta/
models/gemini-2.5-flash:generateContent?key=${GEMINI}`, {method:'post',
contentType:'application/json', payload: JSON.stringify({contents:[{parts:[{text:prompt}]}],
generationConfig:{responseMimeType:'application/
json'}}} )).getContentText()).candidates[0].content.parts[0].text); sh.getRange(r,
4).setValue(o.headline); sh.getRange(r, 5).setValue(o.hook); sh.getRange(r, 6).setValue(o.body);
sh.getRange(r, 7).setValue(o.cta); sh.getRange(r, 8).setValue((o.hashtags||[]).join(' ')); } 강의 시연 포인
트 빈 사이트에서 input 입력 → 메뉴 한 번 클릭 → 30일치 헤드라인이 30초 내 채워지는 모습 한 행 선택 후 "다시 쓰기"
→ 같은 주제의 다른 변주 시연 강의 후 워크숍: 학습자가 자기 비즈니스로 직접 시도 — 결과 비교 토론 트러블슈팅 증상
원인 해결 30건 미만 생성 토큰 부족 maxOutputTokens 8192 유지 + 프롬프트 단순화 날짜가 시작일 이전 AI가 가끔
거꾸로 후처리에서 정렬 + 시작일 검증 추가 채널별 톤 비슷함 프롬프트 강조 부족 채널별 미니 톤 가이드를 pillars에 추
가 JSON 깨짐 따옴표 이스케이프 responseMimeType: 'application/json' 필수 응용 아이디어 이미지 자동 시연:
헤드라인을 받아 DALL·E API로 인스타 카드 이미지 자동 생성 Notion sync: Notion DB로 양방향 동기화 → 팀 협업
A/B 헤드라인: 한 행에 헤드라인 2개 생성 → 게시 후 클릭률 시트로 회수 계절·트렌드 보강: 발행일 기준 검색 트렌드(네
이버 트렌드) 반영하도록 입력 자동화

```

```

{ const pillarLine = i.pillars.length ? `다음 콘텐츠 기동을 균형 있게 분배: ${i.pillars.join(', ')}` : '주제는 자율적으로 다양화'; return `당신은 ${i.business} 도메인의 시니어 콘텐츠 디렉터입니다. 다음 정보로 30일 콘텐츠 캘린더를 작성하세요. 월 테마: ${i.theme} 타겟: ${i.target} 채널: ${i.channels.join(', ')} 톤: ${i.tone} 시작일: ${i.startDate.toISOString().slice(0,10)} ${pillarLine} 규칙: - 각 채널 특성에 맞춰 톤·형식 분기 (예: 블로그=long, 인스타=짧은 후크+이미지 가이드, 링크드인=인사이트, 뉴스레터=주제 묶음) - 같은 주제도 채널별 변주 - 날짜는 시작일부터 30일, 하루에 1~2건 배치 (총 약 30~40건) - 헤드라인은 12~22자, 후크는 1문장, 본문 가이드는 50자 이내, CTA는 명확한 행동 1개, 해시태그 3~6개, 자기과사·과장 금지 JSON 배열만 반환: [{"date":"YYYY-MM-DD","channel":"...", "format":"long|short|image|reel|email", "headline":"...", "hook":"...", "body":"...", "cta":"...", "hashtags":["..."]} `; } function callGemini(prompt) { const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${GEMINI}`; const res = UrlFetchApp.fetch(url, { method: 'post', contentType: 'application/json', payload: JSON.stringify({ contents:[{parts:[{text: prompt}], generationConfig: { responseMimeType: 'application/json', maxOutputTokens: 8192 } }) }); return JSON.parse(JSON.parse(res).getResponse().candidates[0].content.parts[0].text); } function regenerateRow() { const ss = SpreadsheetApp.openById(SHEET_ID); const sh = ss.getSheetByName('calendar'); const r = sh.getActiveCell().getRow(); if (r == 1) return; const row = sh.getRange(r, 1, 1, 9).getValues()[0]; const [date, channel, format, headline] = row; const i = readInputs(); const prompt = `다음 한 건만 새로운 변형으로 다시 작성하세요(같은 날짜·채널·형식 유지). 원본 헤드라인: ${headline} 월 테마: ${i.theme}, 타겟: ${i.target}, 톤: ${i.tone} JSON 단일 객체: { "headline": "...", "hook": "...", "body": "...", "cta": "...", "hashtags": ["..."]} `; const o = JSON.parse(JSON.parse(UrlFetchApp.fetch(`https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-flash:generateContent?key=${GEMINI}`), { method: 'post', contentType: 'application/json', payload: JSON.stringify({ contents:[{parts:[{text: prompt}], generationConfig: { responseMimeType: 'application/json', maxOutputTokens: 8192 } }) })).getResponse().candidates[0].content.parts[0].text); sh.getRange(r, 4).setValue(o.headline); sh.getRange(r, 5).setValue(o.hook); sh.getRange(r, 6).setValue(o.body); sh.getRange(r, 7).setValue(o.cta); sh.getRange(r, 8).setValue(o.hashtags.join(' ')); } const input = readInputs(); const pillars = String(input[0]).split(' ').map(s => s.trim()).filter(Boolean); const channels = String(input[1]).split(' ').map(s => s.trim()).filter(Boolean); const tone = String(input[2]).split(' ').map(s => s.trim()).filter(Boolean); const startDate = new Date(input[3]); const pillars = pillars.map(p => p.trim()).filter(Boolean); const ui = SpreadsheetApp.getUi(); const i = readInputs(); if (!i.theme || !i.channels.length) { ui.alert('input 탭의 월 테마/채널을 먼저 입력하세요.');
```



```

{ const pillarLine = i.pillars.length ? `다음 콘텐츠 기둥을 균형 있게 분배: ${i.pillars.join(', ')}` : '주제는 자
율적으로 다양화'; return `당신은 ${i.business} 도메인의 시니어 콘텐츠 디렉터입니다. 다음 정보로 30일 콘텐츠 캘
린더를 작성하세요. 월 테마: ${i.theme} 타겟: ${i.target} 채널: ${i.channels.join(', ')} 톤: ${i.tone} 시작일: $
{i.startDate.toISOString().slice(0,10)} ${pillarLine} 규칙: - 각 채널 특성에 맞춰 톤·형식 분기 (예: 블로그
=long, 인스타=짧은 후크+이미지 가이드, 링크드인=인사이트, 뉴스레터=주제 묶음) - 같은 주제도 채널별 변주 - 날짜는
시작일: ${i.startDate.toISOString().slice(0,10)}과시·과장 금지 JSON 배열만 반환: [{"date":"YYYY-MM-
${pillarLine}
email","headline":"...","hook":"...","body":"...","cta":"...","hashtags":["..."]} `; } function
규칙allGemini(prompt) { const url = `https://generativelanguage.googleapis.com/v1beta/models/
- 각 채널 특성에 맞춰 톤·형식 분기 (예: 블로그=long, 인스타=짧은 후크+이미지 가이드, 링크드인=인사이트, 뉴
스레터=주제 묶음), contentType:'application/json', payload: JSON.stringify({ contents:[{parts:[{text:
prompt}], responseMimeType:'application/json', maxOutputTokens: 8192} ) } });
- 날짜는 시작일부터 30일 후 하루에 1~2건 배치 (총 약 30~40건) Text().candidates[0].content.parts[0].text);
- 헤드라인은 12~22자, 후크는 1문장, 본문 가이드는 50자 이내, CTA는 명확한 행동 1개, 해시태그 3~6개; const sh =
ss.getSheetByName('calendar'); const r = sh.getActiveCell().getRow(); if (r === 1) return; const row =
sh.getRange(r, 1, 1, 9).getValues()[0]; const [date, channel, format, headline] = row; const i =
readInputs(); const prompt = `다음 한 건만 새로운 변형으로 다시 작성하세요(같은 날짜·채널·형식 유지). 원본 헤
dline: ${headline}, hook: ${hook}, body: ${body}, cta: ${cta}, hashtags: ${hashtags}`; const o =
JSON.parse(JSON.parse(UrlFetchApp.fetch(`https://generativelanguage.googleapis.com/v1beta/
models/gemini-2.5-flash:generateContent?key=${GEMINI}`, {method:'post',
contentType:'application/json', payload: JSON.stringify({contents:[{parts:[{text:prompt}]}],
function callGemini(prompt) {
const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=${GEMINI}`;
const res = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json',
payload: JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:{responseMimeType:'application/json',
maxOutputTokens: 8192}})}).getContentTypeText().candidates[0].content.parts[0].text);
return JSON.parse(JSON.parse(res.getContentTypeText().candidates[0].content.parts[0].text));

function regenerateRow() {
const ss = SpreadsheetApp.openById(SHEET_ID);
const sh = ss.getSheetByName('calendar');
const r = sh.getActiveCell().getRow();
if (r === 1) return;
const row = sh.getRange(r, 1, 1, 9).getValues()[0];
const [date, channel, format, headline] = row;

const i = readInputs();
const prompt = `
다음 한 건만 새로운 변형으로 다시 작성하세요(같은 날짜·채널·형식 유지).
원본 헤드라인: ${headline}
월 테마: ${i.theme}, 타겟: ${i.target}, 톤: ${i.tone}

JSON 단일 객체:
{"headline":"...","hook":"...","body":"...","cta":"...","hashtags":["..."]}
`;
const o = JSON.parse(JSON.parse(UrlFetchApp.fetch(
`https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-
flash:generateContent?key=${GEMINI}`,
{method:'post', contentType:'application/json',
payload: JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:
{responseMimeType:'application/json'}})
).getContentTypeText().candidates[0].content.parts[0].text);

```

```

{ const pillarLine = i.pillars.length ? `다음 콘텐츠 기둥을 균형 있게 분배: ${i.pillars.join(', ')}` : '주제는 자
율적으로 다양화'; return `당신은 ${i.business} 도메인의 시니어 콘텐츠 디렉터입니다. 다음 정보로 30일 콘텐츠 캘
린더를 작성하세요. 월 테마: ${i.theme} 타겟: ${i.target} 채널: ${i.channels.join(', ')} 톤: ${i.tone} 시작일: $
{i.startDate.toISOString().slice(0,10)} ${pillarLine} 규칙: - 각 채널 특성에 맞춰 톤·형식 분기 (예: 블로그
=long, 인스타=짧은 후크+이미지 가이드, 링크드인=인사이트, 뉴스레터=주제 묶음) - 같은 주제도 채널별 변주 - 날짜는
시작일 sh.getRange(r, 4).setValue(o.headline); - 헤드라인은 12~22자, 후크는 1문장, 본문 가이드는 50자 이내,
sh.getRange(r, 5).setValue(o.hook); ~6개 - 자기과시·과장 금지 JSON 배열만 반환: [{"date":"YYYY-MM-
sh.getRange(r, 6).setValue(o.body); DD","channel":"...", "format":"long|short|image|reel|
sh.getRange(r, 7).setValue(o.cta); k":"...", "body":"...", "cta":"...", "hashtags":["..."]} `; } function
sh.getRange(r, 8).setValue((o.hashtags||[]).join(' ')); } const o = UrlFetchApp.fetch(`https://generativelanguage.googleapis.com/v1beta/
models/gemini-2.5-flash:generateContent?key=${GEMINI}`, {method:'post', contentType:'application/json', payload: JSON.stringify({contents:[{parts:[{text:
prompt}], generationConfig:{responseMimeType:'application/json', maxOutputTokens: 8192} } }));
return JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); }
function regenerateRow() { const ss = SpreadsheetApp.openById(SHEET_ID); const sh =
ss.getSheetByName('calendar'); const r = sh.getActiveCell().getRow(); if (r === 1) return; const row =
sh.getRange(r, 1, 1, 9).getValues()[0]; const [date, channel, format, headline] = row; const i =
readingOrder); const prompt = `다음 한 개월 새로운 발행 계획으로 다시 작성하세요 (같은 날짜·채널 형식 유지). 원본 헤
드라인: ${headline} 월 테마: ${i.theme}, 타겟: ${i.target}, 톤: ${i.tone} JSON 단일 객체:
{"headline":"...", "hook":"...", "body":"...", "cta":"...", "hashtags":["..."]} `; const o =
JSON.parse(JSON.parse(UrlFetchApp.fetch(`https://generativelanguage.googleapis.com/v1beta/
models/gemini-2.5-flash:generateContent?key=${GEMINI}`, {method:'post',
contentType:'application/json', payload: JSON.stringify({contents:[{parts:[{text:prompt}],
generationConfig:{responseMimeType:'application/

```

강의 시연 포인트

1. 빈 시트에서 input 입력 → 메뉴 한 번 클릭 → 30일치 헤드라인이 30초 내 채워지는 모습
2. 한 행 선택 후 "다시 쓰기" → 같은 주제와 다른 변주 시연
3. 강의 후 워크숍: 학습자가 자기 비즈니스로 직접 시도 → 결과 비교 토론

트러블슈팅

증상: 빈 시트에서 input 입력 → 메뉴 한 번 클릭 → 30일치 헤드라인이 30초 내 채워지는 모습 한 행 선택 후 "다시 쓰기" 후 같은 주제의 다른 변주 시연 강의 후 워크숍: 학습자가 자기 비즈니스로 직접 시도 → 결과 비교 토론 트러블슈팅 증상 원인 해결

원인: 30초 미만 생성 토큰 부족 maxOutputTokens 8192 유지 + 프롬프트 단순화 날짜가 시작일 이전 시가 가끔 거꾸로 후처리에서 정렬 + 시작일 검증 추가 채널별 톤 비슷함 채널별 미니 톤 가이드를 pillars에 추가 JSON 깨짐 다음표 이스케이프 responseMimeType: 'application/json' 필수 응용 아이디어 이미지 자동 시안: 헤드라인을 받아 DALL·E API로 인스타 카드 이미지 자동 생성 Notion sync: Notion DB로 양방향 동기화 → 팀 협업 A/B 헤드라인: 한 행에 헤드라인 2개 생성 → 게시 후 클릭률 시트로 회수 계절·트렌드 보강: 발행일 기준 검색 트렌드(네이버 트렌드) 반영하도록 입력 자동화

JSON 깨짐

다음표 이스케이프

responseMimeType: 'application/json' 필수

응용 아이디어

- **이미지 자동 시안**: 헤드라인을 받아 DALL·E API로 인스타 카드 이미지 자동 생성
- **Notion sync**: Notion DB로 양방향 동기화 → 팀 협업
- **A/B 헤드라인**: 한 행에 헤드라인 2개 생성 → 게시 후 클릭률 시트로 회수
- **계절·트렌드 보강**: 발행일 기준 검색 트렌드(네이버 트렌드) 반영하도록 입력 자동화

```
MKT-02: {lead.name} 직위: ${lead.jobTitle}, 직역: ${lead.jobCount / 매달 $1  
[lead.ranking]} 위력 내용: "${lead.content}" | Price(0,4000) | "" | 유무: - Score:
```

난이도

예상 소요

Google Form +

0원

Gemini API 키 + Slack

Bot

한 줄 핵심: 문의 품이 들어오면 시가 회사:문의 내용을 보고 접수 (Hot/Warm/Cold)를 매기고, 그에 따라 담당자에게 자동 배정 알림한다. elements: [{type: 'mrkdwn', text: `담당자: \${assignee.name}`}] }

왜 이 자동화인가

항목

수동(Before)

자동(After)

-\$\{fromName\$
첫 응답까지 시간

평균 6~24시간

2~10분

무.예산만 다를 때

갈리는 주관적

일과 관련된 AI

Hot 리드 누락

조조 발새

의면 5
거의 0

저화율 영향

응답시간 5분 이내일 때 21배 ↑ (HBR)

적용 시나리오 - B2B 컨설팅 - SaaS 분석: 양면의 워크플로 보강

주요 내용: Zapier를 통해 HubSpot CRM 연동, Gmail 연동, Slack 연동, Twilio 연동, Calendly 연동, Stripe 연동, Mailchimp 연동, Salesforce 연동, HubSpot Spot/세일즈포스 API로 리드 자동 생성 (Apps Script가 webhook으로 push) 이메일 인박스 분석: Gmail 라벨로 들어온 새 메일을 같은 로직으로 스코어링 다국어 자동 응답: 품 응답이 영어이면 영어 템플릿으로 분기 No-Response 리텐션: 14일간 연락 없는 리드에 자동 follow-up 메일

- Google Form (문의)
- Google Sheet (응답 + 롤 + 담당자)
- Gemini API (스코어링 보강)
- Slack Webhook (담당자별 채널)
- Apps Script (Form 제출 트리거)

1. 회사/소속


```

    (cond.startsWith('직원=')) return hc === cond.slice(3); if (cond === '예산>=1억') return budget === '1억
    +'; if (cond === '예산 5천~1억') return budget === '5천~1억'; if (cond === '예산 1천~5천') return budget
    === '1천~5천'; if (cond === '직원 200+') return hc === '200+'; if (cond.startsWith('시작=')) return
    timing === cond.slice(3); return false; } function aiAssess(lead) { const prompt = `당신은 B2B 세일즈
    코치입니다. 다음 리드의 실제 구매 의향과 적합성을 평가하세요. 리드 정보: - 회사: ${lead.company} - 이름: $
    ${lead.name} - 직위: ${lead.jobTitle} / 직원: ${lead.headcount} / 예산: ${lead.budget} / 시작: $
    ${lead.timing} - 의뢰 내용: "${(lead.content||'').slice(0, 4000)}" 규칙: - score: 0~100 점수 (구매 의향 +
    우리 솔루션 적합성) - comment: 한 문장, 영업 담당자에게 도움될 핵심 한 마디 - urgent: true/false (24시간 내 응
    답 권고 여부) JSON: {"score":int,"comment":"...","urgent":bool}`; const url = `https://
    generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=$
    {GEMINI}`; const res = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json',
    payload: JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:
    {responseMimeType:'application/json'}})); return
    JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
    sendSlackCard(assignee, lead) { const channel = assignee.userId || assignee.channel; const emoji =
    lead.grade === 'HOT' ? '🔥' : '⚠️'; const body = { channel, text: `${emoji} ${lead.grade} 리드 인입 — $
    ${lead.company}`, blocks: [ {type:'header', text:{type:'plain_text', text:`${emoji} ${lead.grade} 리드 —
    ${lead.score}점`}, {type:'section', fields:[ {type:'mrkdwn', text: `*회사*\n${lead.company}`,
    {type:'mrkdwn', text: `*이름/직무*\n${lead.name} / ${lead.jobTitle}`, {type:'mrkdwn', text: `*이메일
    *\n${lead.email}`, {type:'mrkdwn', text: `*점수*\n${lead.score}/100`, }], {type:'section', text:
    {type:'mrkdwn', text: `*AI 코멘트*\n${lead.ai.comment}\n${lead.ai.urgent ? ':alarm_clock: 24시간 내 응
    답 권고' : ''}}}, {type:'section', text:{type:'mrkdwn', text: `*의뢰 요약*\n${(lead.content||'').slice(0,
    600)}}}, {type:'context', elements:[{type:'mrkdwn', text: `담당자: *${assignee.name}*`}] } ] }; if
    (SLACK_TOK) { UrlFetchApp.fetch('https://slack.com/api/chat.postMessage', { method:'post',
    contentType:'application/json; charset=utf-8', headers:{Authorization: `Bearer ${SLACK_TOK}`},
    payload: JSON.stringify(body) }); } } function autoReplyToProspect(email, name, fromName) { const
    html = `

<p>$
    {name||'고객'} 님, 문의 감사드립니다.</p> <p>담당자가 영업일 기준 24시간 안에 직접 연락드리겠습니다. 그 사이 아
    래 자료를 먼저 보시면 도움이 됩니다.</p> <ul> <li><a href="https://www.nedabah.org/cases.html">사례 모
    음</a></li> <li><a href="https://www.nedabah.org/lectures/">강의/워크숍 안내</a></li> </ul> <p
    style="color:#6a604f;">— ${fromName}</p> </div>`; GmailApp.sendEmail(email, `문의 잘 받았습니
    다 — ${fromName}`, '', {htmlBody: html, name: fromName}); } 강의 시연 포인트 폼 제출 → 1분 내 점수 계산 +
    담당자 Slack 카드 + 자동 응답 메일 시트 rules 한 줄을 수정해 점수 즉시 변경되는 모습 (코드 0줄 수정) 같은 의뢰가
    직무·예산만 다를 때 등급이 어떻게 갈리는지 비교 트러블슈팅 증상 원인 해결 점수가 0 룰 조건 문자열 불일치 rules 시트
    의 라벨을 폼 보기 그대로 복사 Slack 카드 미수신 채널에 Bot 미초대 /invite @봇이름 AI가 너무 후함 프롬프트의 "적
    합성" 강조 부족 "우리 솔루션과 맞지 않으면 50 미만" 한 줄 추가 자동 응답이 누락 Gmail 송신 실패
    MailApp.sendEmail로 대체 가능 안전·윤리 메모 본 자동화의 aiAssess는 회사명·이름·의뢰 내용을 Gemini에 전송합
    니다. B2B 영업 적합성 평가라는 명확한 비즈니스 목적이 있으므로 PII 위반에 해당하지 않습니다 — 개인 응답 분석과는
    다른 맥락이라는 점이 중요합니다. 비교: HR 펄스서베이(가이드 HR-03)는 개인 익명 응답이므로 식별 정보를 AI에 절대
    전송하지 않도록 별도 설계되어 있습니다. 강의에서는 이 두 케이스를 함께 보여 "어떤 데이터를 AI에 보낼지의 판단 기
    준"을 토론합니다. 응용 아이디어 CRM 연동: HubSpot/세일즈포스 API로 리드 자동 생성 (Apps Script가
    webhook으로 push) 이메일 인박스 분석: Gmail 라벨로 들어온 새 메일을 같은 로직으로 스코어링 다국어 자동 응답:
    폼 응답이 영어이면 영어 템플릿으로 분기 No-Response 리텐션: 14일간 연락 없는 리드에 자동 follow-up 메일


```

```

    (cond.startsWith('직원=')) return hc === cond.slice(3); if (cond === '예산>=1억') return budget === '1억
    +'; if (cond === '예산 5천~1억') return budget === '5천~1억'; if (cond === '예산 1천~5천') return budget
    === '1천~5천'; if (cond === '직원 200+') return hc === '200+'; if (cond.startsWith('시작=')) return
    timing === cond.slice(3); return false; } function aiAssess(lead) { const prompt = `당신은 B2B 세일즈
    코치입니다. 다음 리드의 실제 구매 의향과 적합성을 평가하세요. 리드 정보: - 회사: ${lead.company} - 이름: $
    {lead.name} - 이메일: ${lead.email} - 직위: ${lead.jobTitle} - 직급: ${lead.headcount} / 예산: ${lead.budget} / 시작: $
    {lead.timing} - 리드 번호: "${lead.content.slice(0, 100)}" 규칙: - score: 0~100 점수 (구매 의향 +
    우량 솔루션 적합성) - cond: '직무'는 문장 앞의 대문자 앞에 도음칠 핵심 한 마디 - urgent: true/false (24시간 내 응
    답 권고 여부) - Slack: 'SLACK_TOK' ("score: 0~100", "urgent: bool") `; const url = `https://
    generativeai.googleapis.com/v1beta1/models/gemini-2.5-flash:generateContent?key=$
    {GEMINI}`; const res = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json',
    payload: JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:
    {responseMimeType:'application/json'}})); return
    JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
    sendSlackCard(assignee, lead, channel, assigneeId, assigneeName) { const emoji =
    lead.grade === 'HOT' ? '🔥' : '🟡'; const body = {channel: channel, text: `${emoji} ${lead.grade} 리드 -
    ${lead.company}`, blocks: [ {type:'header', text:{type:'plain_text', text: `${emoji} ${lead.grade} 리드 -
    ${lead.company}`, {type:'section', fields:[ {type:'mrkdwn', text: `회사*\\n${lead.company}`,
    {type:'mrkdwn', text: `이름(직무)*\\n${lead.name}, / ${lead.jobTitle}`, {type:'mrkdwn', text: `이메일
    [0]:*\\n${lead.email}`, {type:'mrkdwn', text: `점수*\\n${lead.score}/100`, }, {type:'section', text:
    {type:'mrkdwn', text: `AI 코멘트*\\n${lead.ai.comment}\\n${lead.urgent ? 'alarm clock: 24시간 내 응
    답 권고 : '1'} `, {type:'section', text:{type:'mrkdwn', text: `의뢰 요약*\\n${(lead.content||'').slice(0,
    60)} ` }, {type:'context', elements:[{type:'mrkdwn', text: `담당자: *${assignee.name}*` } ] } }; if
    (SLACK_TOK) { UrlFetchApp.fetch(`https://slack.com/api/chat.postMessage`, { method:'post',
    contentType:'application/json'; charset=utf-8; headers:{Authorization: `Bearer ${SLACK_TOK}`},
    payload: JSON.stringify(body) }); } } function autoReplyToProspect(email, name, fromName) { const
    html = `<div style="font-family: Noto Sans KR, sans-serif; line-height: 1.7; max-width: 600px;"> <p>${
    name}</p> <p>귀하의 문의 감사드립니다.</p> <p>담당자가 영업일 기준 24시간 안에 직접 연락드리겠습니다. 그 사이 하
    래 자료를 먼저 보시면 도움이 됩니다.</p> <ul> <li><a href="https://www.nedabah.org/cases.html">사례 모
    음</a></li> <li><a href="https://www.nedabah.org/lectures/">강의/워크숍 안내</a></li> </ul> <p
    style="color: #66604f;">— ${fromName}</p> </div>`; GmailApp.sendEmail(email, `문의를 잘 받았습니다
    — ${fromName}`, {htmlBody: html, name: fromName}); } 강의 시연 포인트 폼 제출 → 1분 내 점수 계산 +
    담당자 Slack 카드 + 자동 응답 메일 시트 rules 한 줄을 수정해 점수 즉시 변경되는 모습 (코드 0줄 수정) 같은 의미가
    직무·예산만 다를 때 등급이 어떻게 갈리는지 비교 드러블스틱 증산 원인 해결 점수가 0를 조건 문자열 붙일치 rules 시트
    의 라벨을 폼 보기 그대로 복사 Slack 카드 미수신 채널에 Bot 미초대 /invite @봇이름 AI가 너무 후함 프롬프트의 "적
    함성" 강조 부족 "우리 솔루션과 맞지 않으면 50 미만" 한 줄 추가 자동 응답이 누락 Gmail 송신 실패
    MailApp.sendEmail로 대체 가능 안전: 윤리 메모 본 자동화의 aiAssess는 회사명·이름·의뢰 내용을 Gemini에 전송합
    니다. B2B 영업 적합성 평가라는 명확한 비즈니스 목적이 있으므로 PII 외박에 해당하지 않습니다. — 개인 응답 분석과는
    다른 맥락이라는 점이 중요합니다. 비교: HR 필수서비스(가이드 HR-03)는 개인 익명 응답이므로 식별 정보를 AI에 절대
    전송하지 않도록 별도 설계되어 있습니다. 강의에서는 이 두 케이스를 함께 보여 "어떤 데이터를 AI에 보낼지의 판단 기
    준을 늘렸습니다. 응용 아이디어 CRM 연동: HubSpot/세일즈포스 API로 리드 자동 생성 (Apps Script가
    webhook으로 push) 이메일 외박스 분석: Gmail 라벨로 들어온 새 메일을 같은 로직으로 스코어링 다국어 자동 응답:
    폼 응답이 영어이면 영어 웹플릿으로 보기 NoResponse 리텐션: 14일간 연락 없는 리드에 자동 follow-up 메일
    autoReplyToProspect(email, name, FROM);
    } else {
    autoReplyToProspect(email, name, FROM);
    }

    // 5) 로그 적재
    const log = ss.getSheetByName('log');
    log.appendRow([new Date(), company, score, grade, assignee ? assignee.name : '자동응대',
    ai.comment]);
    }

    function matchesCond(cond, job, hc, budget, timing) {
    cond = String(cond).trim();
    // 정확한 옵션 문자열만 매칭한다. 부분문자열 매칭은 절대 사용하지 않는다
    // (예: budget이 '5천~1억'이면 '예산>=1억' 룰이 부분일치로 잘못 발동되어 점수가 이중 가산되는 버그)
    if (cond.startsWith('직무=')) return job === cond.slice(3);
    if (cond.startsWith('직원=')) return hc === cond.slice(3);
    if (cond === '예산>=1억') return budget === '1억+';
    if (cond === '예산 5천~1억') return budget === '5천~1억';
    if (cond === '예산 1천~5천') return budget === '1천~5천';
  
```

```

    (cond.startsWith('직원=')) return hc === cond.slice(3); if (cond === '예산>=1억') return budget === '1억
    +'; if (cond === '예산 5천~1억') return budget === '5천~1억'; if (cond === '예산 1천~5천') return budget
    === '1천~5천'; if (cond === '직원 200+') return hc === '200+'; if (cond.startsWith('시작=')) return
    timing === cond.slice(3); return false; } function aiAssess(lead) { const prompt = `당신은 B2B 세일즈
    코치입니다. 다음 리드의 실제 구매 의향과 적합성을 평가하세요. 리드 정보: - 회사: ${lead.company} - 이름: $
    {lead.name} - 직위: ${lead.jobTitle} / 직원: ${lead.headcount} / 예산: ${lead.budget} / 시작: $
    {lead.timing} - 의뢰 내용: "${(lead.content||'').slice(0, 4000)}" 규칙: - score: 0~100 점수 (구매 의향 +
    우리 솔루션 적합성) - comment: 한 문장, 영업 담당자에게 도움될 핵심 한 마디 - urgent: true/false (24시간 내 응
    답 권고 여부) JSON: {"score":int,"comment":"...","urgent":bool}`; const url = `https://
    generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=$
    {GEMINI}`; const res = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json',
    const prompt = `당신은 B2B 세일즈 코치입니다. 다음 리드의 실제 구매 의향과 적합성을 평가하세요.
    JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function
    sendSlackCard(assignee, lead) { const channel = assignee.userId || assignee.channel; const emoji =
    lead.grade === 'HOT' ? '🔥' : '⚠️'; const body = { channel, text: `${emoji} ${lead.grade} 리드 인입 - $
    {lead.company}`, blocks: [ {type:'header', text:{type:'plain_text', text:`${emoji} ${lead.grade} 리드 -
    - 직위: ${lead.jobTitle} / 직원: ${lead.headcount} / 예산: ${lead.budget} / 시작: ${lead.timing}
    - 의뢰 내용: "${(lead.content||'').slice(0, 4000)}" }, {type:'section', fields:[ {type:'mrkdwn', text: `회사: ${lead.company}`,
    {type:'mrkdwn', text: `이름/직위: ${lead.name} / ${lead.jobTitle}`, {type:'mrkdwn', text: `이메일
    *${lead.email}`, {type:'mrkdwn', text: `점수* ${lead.score}/100` }, ], {type:'section', text:
    {type:'mrkdwn', text: `*AI 코멘트* ${lead.ai.comment}\n${lead.ai.urgent ? ':alarm_clock: 24시간 내 응
    답 권고 : ' : ''}}}, {type:'section', text:{type:'mrkdwn', text: `*의뢰 요약* ${(lead.content||'').slice(0,
    600)}}}, {type:'text', elements:[{type:'mrkdwn', text: `담당자: ${assignee.name}*`}} ] }]; if
    (SLACK_TOK) { UrlFetchApp.fetch(`https://slack.com/api/chat.postMessage`, { method:'post',
    contentType:'application/json; charset=utf-8', headers:{Authorization:`Bearer ${SLACK_TOK}`},
    payload: JSON.stringify(body) }); } } function autoReplyToProspect(email, name, fromName) { const
    html = `<div style="font-family:'Noto Sans KR', sans-serif; line-height:1.7; max-width:600px;"><p>${
    {name}}</p><p>문의 감사드립니다.</p><p>담당자가 응답할 기준 24시간 안에 직접 연락드리겠습니다. 그 사이 하
    래 자료를 먼저 보시면 도움이 됩니다.</p><ul><li><a href="https://www.nedabah.org/cases.html">사례 모
    음</a></li><li><a href="https://www.nedabah.org/lectures/">강의/워크숍 안내</a></li></ul><p>
    style="color:#66b044;">- ${fromName}</p></div>`; GmailApp.sendEmail(email, '문의할 잘 받았습니다
    - ${fromName}', '', {htmlBody: html, name: fromName}); } 강의 시연 포인트 폼 제출 → 1분 내 점수 계산 +
    담당자 Slack 카드 + 자동 응답 메일 시트 rules 한 줄을 수정해 점수 즉시 변경되는 모습 (코드 0줄 수정) 같은 의뢰가
    직무-예산만 다를 때 등급이 어떻게 갈리는지 비교 트러블슈팅 증상 원인 해결 점수가 0 를 조건 문자열 불일치 rules 시트
    의 라벨을 폼 보기 그대로 복사 Slack 카드 미수신 채널에 Bot 미초대 /invite @뭇이름 시가 미수 후임 프롬프트의 "적
    합성" 강조 부족 "우리 솔루션과 맞지 않으면 50 미만" 한 줄 추가 자동 응답이 누락 Gmail 송신 실패
    MailApp.sendEmail로 대체 가능 안전·윤리 메모 본 자동화의 aiAssess는 회사명·이름·의뢰 내용을 Gemini에 전송합
    니다. B2B 영업 적합성 평가라는 명확한 버즈디스 목적에 있으므로 PII 위반에 해당하지 않습니다 - 개인 응답 분석과는
    다른 맥락이라는 점이 중요합니다. 비교: HR 필수 이메일 가이드 (HR-03)는 개인 익명 응답이므로 식별 정보를 AI에 절대
    전송하지 않도록 철저히 설계되어 있습니다. 강의에서는 이 두 케이스를 함께 보여 "어떤 데이터를 AI에 보낼지의 판단 기
    준"을 토론합니다. 응용 아이디어 CRM 연동: HubSpot/세일즈포스 API로 리드 자동 생성 (Apps Script가
    webhook으로 push) 이메일 인박스 분석: Gmail 라벨로 들어온 새 메일을 같은 로직으로 스코어링 다국어 자동 응답:
    폼 응답이 영어이면 영어, 폼을 읽기로 불가 No Response 라면엔 14일간 연락 없는 리드에 자동 follow-up 메일
    blocks: [
    {type:'header', text:{type:'plain_text', text:`${emoji} ${lead.grade} 리드 - $
    {lead.score}점}},
    {type:'section', fields:[
    {type:'mrkdwn', text: `*회사* ${lead.company}`,
    {type:'mrkdwn', text: `*이름/직위* ${lead.name} / ${lead.jobTitle}`,
    {type:'mrkdwn', text: `*이메일* ${lead.email}`,
    {type:'mrkdwn', text: `*점수* ${lead.score}/100`,
    ]},
    {type:'section', text:{type:'mrkdwn', text: `*AI 코멘트* ${lead.ai.comment}\n$
    {lead.ai.urgent ? ':alarm_clock: 24시간 내 응답 권고 : ' : ''}}},
    {type:'section', text:{type:'mrkdwn', text: `*의뢰 요약*\n$
    {(lead.content||'').slice(0, 600)}}},
    {type:'text', elements:[{type:'mrkdwn', text: `담당자: ${assignee.name}*`}}
    ]
    };
    if (SLACK_TOK) {
    UrlFetchApp.fetch('https://slack.com/api/chat.postMessage', {
    method:'post', contentType:'application/json; charset=utf-8',
  
```


되어 점수가 최종 가산되는 버그) if (cond.startsWith('직원= ')) return job === cond.slice(3); if (cond.startsWith('직원=') return hc === cond.slice(3); if (cond === '예산>=1억') return budget === '1억 +'; if (cond === '예산 5천~1억') return budget === '5천~1억'; if (cond === '예산 1천~5천') return budget === '1천~5천'; if (cond === '직원 200+') return hc === '200+'; if (cond.startsWith('시작=')) return timing === cond.slice(3); return false; } function aiAssess(lead) { const prompt = `당신은 B2B 세일즈 코치입니다. 다음 리드의 실제 구매 의향과 적합성을 평가하세요. 리드 정보: - 회사: \${lead.company} - 이름: \${lead.name} - 직무: \${lead.job_title} - 직위: \${lead.title} - 필요 count: \${lead.need_count} - 예산: \${lead.budget} - 라벨: \${lead.label} - 필요 내용: \${lead.content} - AI 코멘트: \${lead.ai_comment} - AI urgent: \${lead.ai_urgent} - 구매 의향: \${lead.purchase_intent} - 우리 솔루션 적합성: \${lead.solution_fit} - comment: 한 문장, 영업 담당자에게 도움될 핵심 한 마디 - urgent: true/false (24시간 내 응답 권고 여부) JSON: {"score":int,"comment":"...","urgent":bool}`; const url = `https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=\${GEMINI}`; const res = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json', payload: JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:{responseMimeType:'application/json'}})); return JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } function sendSlackCard(assignee, lead) { const channel = assignee.userId || assignee.channel; const emoji = lead.label ? `🔖 \${lead.label}` : `📧`; const body = `📧 \${lead.name} | \${lead.job\_title} | \${lead.title} | \${lead.need\_count} | \${lead.budget} | \${lead.label} | \${lead.content} | \${lead.ai\_comment} | \${lead.ai\_urgent} | \${lead.purchase\_intent} | \${lead.solution\_fit} | \${lead.comment} | \${lead.ai\_score}`; const blocks = [{type:'header', text:`\${emoji} \${lead.name} | \${lead.job\_title} | \${lead.title} | \${lead.need\_count} | \${lead.budget} | \${lead.label} | \${lead.content} | \${lead.ai\_comment} | \${lead.ai\_urgent} | \${lead.purchase\_intent} | \${lead.solution\_fit} | \${lead.comment} | \${lead.ai\_score}`, {type:'section', text:`회사\*\${lead.company}`, {type:'mrkdwn', text:`이메일\*\${lead.email}`, {type:'mrkdwn', text:`점수\*\${lead.score}/100`, {type:'section', text:`AI 코멘트\*\${lead.ai\_comment}\${lead.ai\_urgent ? ':alarm\_clock: 24시간 내 응답 권고' : ''}`, {type:'section', text:`의뢰 요약\*\${(lead.content||'').slice(0, 600)}`, {type:'context', elements:[{type:'mrkdwn', text:`담당자: \*\${assignee.name}\*`}] }] }; if (SLACK_TOK) { UrlFetchApp.fetch('https://slack.com/api/chat.postMessage', { method:'post', contentType:'application/json; charset=utf-8', headers:{Authorization:`Bearer \${SLACK\_TOK}`}, payload: JSON.stringify(body) }); } } function autoReplyToProspect(email, name, fromName) { const html = `

<p>\${name||'고객'} 님, 문의 감사드립니다.</p> <p>담당자가 영업일 기준 24시간 안에 직접 연락드리겠습니다. 그 사이 아래 자료를 먼저 보시면 도움이 됩니다.</p> 사례 모음 강의/워크숍 안내 <p style="color:#6a604f;">— \${fromName}</p> </div>`; GmailApp.sendEmail(email, `문의 잘 받았습니다 - \${fromName}`, '', {htmlBody: html, name: fromName}); } 강의 시연 포인트 폼 제출 → 1분 내 점수 계산 + 담당자 Slack 카드 + 자동 응답 메일 시트 rules 한 줄을 수정해 점수 즉시 변경되는 모습 (코드 0줄 수정) 같은 의뢰가 직무·예산만 다를 때 등급이 어떻게 갈리는지 비교 트러블슈팅 증상 원인 해결 점수가 0 를 조건 문자열 붙일지 rules 시트의 라벨을 폼 보기 그대로 복사 Slack 카드 미수신 채널에 Bot 미초대 /invite @봇이름 AI가 너무 후함 프롬프트의 "적합성" 강조 부족 "우리 솔루션과 맞지 않으면 50 미만" 한 줄 추가 자동 응답이 누락 Gmail 송신 실패 MailApp.sendEmail로 대체 가능 안전·윤리 메모 본 자동화의 aiAssess는 회사명·이름·의뢰 내용을 Gemini에 전송합니다. B2B 영업 적합성 평가라는 명확한 비즈니스 목적이 있으므로 PII 위반에 해당하지 않습니다 — 개인 응답 분석과는 다른 맥락이라는 점이 중요합니다. 비교: HR 펄스서베이(가이드 HR-03)는 개인 익명 응답이므로 식별 정보를 AI에 절대 전송하지 않도록 별도 설계되어 있습니다. 강의에서는 이 두 케이스를 함께 보여 "어떤 데이터를 AI에 보낼지의 판단 기준"을 토론탐니다. 응용 아이디어 CRM 연동: HubSpot/세일즈포스 API로 리드 자동 생성 (Apps Script가 webhook으로 push) 이메일 인박스 분석: Gmail 라벨로 들어온 새 메일을 같은 로직으로 스코어링 다국어 자동 응답: 폼 응답이 영어이면 영어 템플릿으로 분기 No-Response 리텐션: 14일간 연락 없는 리드에 자동 follow-up 메일

응용 아이디어

- CRM 연동: HubSpot/세일즈포스 API로 리드 자동 생성 (Apps Script가 webhook으로 push)

- 이메일 인박스 분석: Gmail 라벨로 들어온 새 메일을 같은 로직으로 스코어링

- 다국어 자동 응답: 폼 응답이 영어이면 영어 템플릿으로 분기

- No-Response 리텐션: 14일간 연락 없는 리드에 자동 follow-up 메일

```
{responseMimeType:'application/json'}}) }); return
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } //
// 2) 주간 리
```

포트 //

```
function weeklyReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const all =
ss.getSheetByName('mentions').getDataRange().getValues(); const head = all[0]; const since = new
Date(Date.now() - 7*86400000); const week = all.slice(1).filter(r => new Date(r[0]) >= since); if
(week.length === 0) return; const grouped = {}; let pos=0, neg=0, neu=0; const topicCount = {};
week.forEach(r => { const [date, keyword, source, title, url, snippet, sentiment, topics] = r;
```

```
grouped[keyword] = grouped[keyword] || {pos:0, neg:0, neu:0, items:[]};
grouped[keyword].items.push({title, url, sentiment, topics, snippet}); if (sentiment === 'positive')
{pos++; grouped[keyword].pos++;} else if (sentiment === 'negative') {neg++; grouped[keyword].neg+
+;} else {neu++; grouped[keyword].neu++;} String(topics||'').split(',').filter(Boolean).forEach(t =>
topicCount[t.trim()] = (topicCount[t.trim()]||0)+1); }); const topTopics =
★★Object.entries(topicCount).sort((a,b)=>b[1]-a[1]); const insight = aiInsight({total:
week.length, pos, neg, neu, topTopics, grouped}); Gemini API 키
topTopics, grouped, insight); if (EMAIL_TO) GmailApp.sendEmail(EMAIL_TO, `[주간] 브랜드 멘션 다이제
스트 — ${new Date().toISOString().slice(0,10)}`, '', {htmlBody: html});
```

```
ss.getSheetByName('weekly_report').appendRow([new Date(), week.length, pos, neg, neu,
JSON.stringify(insight)]); } function aiInsight(d) { const ctx = d.topTopics.map([t,c])=>` ${t}($
{c})`; const prompt = `브랜드 매니저로서 브랜드 멘션 주간 리포트 작성. 한 줄로 200자 이내. 네가
시지를 분류한 1개의 키워드와 함께 말해줘. pos / 부정 ${d.neg} / 중립 ${d.neu} - 상위 주제: ${ctx} JSON:
{summary:"...",watch:"...",action:"..."} - summary: 한 문장 종합 - watch: 다음 주 주의해서 지켜볼 한 가
지 - action: 마케팅·CS·제품팀 중 한 곳에 보낼 권고 한 줄`; const url = `https://
```

```
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=$
{GEMINI}`; const r = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json', payload:
JSON.stringify({contents:[{parts:[{text:prompt}], generationConfig:
{responseMimeType:'application/json'}})})); return
```

```
JSON.parse(JSON.parse(r.getContentText()).candidates[0].content.parts[0].text); } function
render(total, sent, topTopics, grouped, ins) { const sumTotal = sent.pos + sent.neg + sent.neu || 1;
const topicHtml = topTopics.map([t,c])=>`<li><b>${t}</b> · ${c}</li>`.join(''); let kwHtml = "";
for (const k in grouped) { const g = grouped[k]; const negs =
g.items.filter(it=>it.sentiment==='negative').slice(0,3); const negHtml = negs.length ? `<ul>${
negs.map(n=>`<li><a href="${n.url}">${n.title}</a></li>`.join('')}</ul>` : `<p style="color:#888;">
부정 멘션 없음</p>`; kwHtml += `<h4>${k}</h4><p>긍정 ${g.pos} / 부정 ${g.neg} / 중립 ${g.neu}</p> <p
style="color:#b45309;"><b>주의 멘션</b></p>${negHtml}`; } return `<div style="font-family:'Noto
Sans KR', sans-serif;line-height:1.7;max-width:740px;color:#222;"><h2 style="border-bottom:2px
solid #b45309;padding-bottom:.5rem;">브랜드 멘션 주간 다이제스트</h2> <p style="color:#6a604f;">${
[new Date().toLocaleDateString('ko-KR')] · 총 ${total}건</p> <h3>전체 감성</h3> <p>긍정 ${((sent.pos/
sumTotal*100).toFixed(0))}% / 부정 ${((sent.neg/sumTotal*100).toFixed(0))}% / 중립 ${((sent.neu/
sumTotal*100).toFixed(0))}%</p> <h3>상위 주제</h3> <ul>${topicHtml}</ul> <h3>🔴 핵심 인사이트</h3>
<p><b>요약 — </b>${ins.summary}</p> <p><b>다음 주 관찰 — </b>${ins.watch}</p> <p><b>권고 액션 —
</b>${ins.action}</p> <hr> <h3>키워드별 상세</h3> ${kwHtml} <p style="color:#999;font-size:.
85rem;margin-top:2rem;">자동 생성 · 출처: 네이버 블로그/카페/뉴스</p> </div>`; } 강의 시연 포인트
```

```
keywords 시트에 자사 키워드 한 줄 추가 → dailyCollect() 즉시 실행 → 시트 채워지는 모습 weeklyReport() 즉시
실행 → 이메일 도착 (블로그/카페 무료 월 25,000건) client id, client secret 필요 → 위기관리 흐름 토
론 트러블슈팅 증상 원인 해결 네이버 API 401 Key 누락/오타 콘솔에서 새 Application 등록 확인 일 25,000 한도 초
과 → Google News RSS (보강)
```

```
• Google Sheet (수집 + 분석)
• Google API (검색)
• Gemini API (분석)
• Apps Script (일별 수집 트리거 + 주간 리포트 트리거)
```

셋업 가이드

Step 1. 네이버 개발자 센터 키 발급

- <https://developers.naver.com/main/> → 애플리케이션 등록
- "검색" API 사용 신청 → Client ID, Client Secret 복사

```

    {responseMimeType:'application/json'}})); return
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } //
----- // 2) 주간 리
포트 // -----
function weeklyReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const all =
ss.getSheetByName('mentions').getDataRange().getValues(); const head = all[0]; const since = new
Date(Date.now() - 7*86400000); const week = all.slice(1).filter(r => new Date(r[0]) >= since); if
(week.length === 0) return; const grouped = {}; let pos=0, neg=0, neu=0; const topicCount = {};
week.forEach(r => { const [date, keyword, source, title, url, snippet, sentiment, topics] = r;
| 본문스니펫 | 감성 | 주제 | 탭 weekly_report; 누적 리포트 전체
grouped[keyword] = grouped[keyword] || {pos:0, neg:0, neu:0, items:[]};
grouped[keyword].items.push({title, url, sentiment, topics, snippet}); if (sentiment === 'positive')
{pos++; grouped[keyword].pos++;} else if (sentiment === 'negative') {neg++; grouped[keyword].neg+
} else {neu++; grouped[keyword].neu++;} String(topics||'').split(',').filter(Boolean).forEach(t =>
topicCount[t.trim()] = (topicCount[t.trim()]||0)+1); }); const topTopics =
Object.entries(topicCount).sort((a,b)=>b[1]-a[1]).slice(0, 8); const insight = aiInsight({total:
week.length, pos, neg, neu, topTopics, grouped}); const html = render(week.length, {pos,neg,neu},
topTopics, grouped, insight); if (EMAIL_TO) GmailApp.sendEmail(EMAIL_TO, `[주간] 브랜드 멘션 다이제
스트 — ${new Date().toISOString().slice(0,10)}`, '{htmlBody: html}');
ss.getSheetByName('weekly_report').appendRow([new Date(), week.length, pos, neg, neu,
JSON.stringify(insight)]); } function aiInsight(d) { const ctx = d.topTopics.map(([t,c])=>`${t}($
{c})`); const prompt = `브랜드 매니저용 주간 코멘트 작성. 한국어 250자 이내. 데이터: - 총 멘션: $
{d.total} - 감성: 긍정 ${d.pos} / 부정 ${d.neg} / 중립 ${d.neu} - 상위 주제: ${ctx} JSON:
{"summary":"...", "watch":"...", "action":"..."} - summary: 한 문장 종합 - watch: 다음 주 주의해서 지켜볼 한 가
지 - action: 마케팅·CS·제품팀 중 한 곳에 보낼 권고 한 줄`; const url = `https://
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=$
{GEMINI}`; const r = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json', payload:
JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:
{responseMimeType:'application/json'}})); return
JSON.parse(JSON.parse(r.getContentText()).candidates[0].content.parts[0].text); } function
render(total, sent, topTopics, grouped, ins) { const sumTotal = sent.pos + sent.neg + sent.neu || 1;
const topicHtml = topTopics.map(([t,c])=>`<li><b>${t}</b> · ${c}건</li>`).join(''); let kwHtml = '';
for (const k in grouped) { const g = grouped[k]; const negs =
g.items.filter(it=>it.sentiment==='negative').slice(0,3); const negHtml = negs.length ? `<ul>${
negs.map(n=>`<li><a href="${n.url}">${n.title}</a></li>`).join('')}</ul>` : `<p style="color:#888;">
부정 멘션 없음</p>`; kwHtml += `<h4>${k}</h4> <p>긍정 ${g.pos} / 부정 ${g.neg} / 중립 ${g.neu}</p> <p
style="color:#b45309;"><b>주의 멘션</b></p>${negHtml}`; } return `<div style="font-family:'Noto
Sans KR',sans-serif;line-height:1.7;max-width:740px;color:#222;"> <h2 style="border-bottom:2px
solid #b45309;padding-bottom:.5rem;">브랜드 멘션 주간 다이제스트</h2> <p style="color:#6a604f;">${
new Date().toLocaleDateString('ko-KR')} · 총 ${total}건</p> <h3>전체 감성</h3> <p>긍정 ${((sent.pos/
sumTotal*100).toFixed(0))}% / 부정 ${((sent.neg/sumTotal*100).toFixed(0))}% / 중립 ${((sent.neu/
sumTotal*100).toFixed(0))}%</p> <h3>상위 주제</h3> <ul>${topicHtml}</ul> <h3>🔴 핵심 인사이트</h3>
<p><b>요약 — </b>${ins.summary}</p> <p><b>다음 주 관찰 — </b>${ins.watch}</p> <p><b>권고 액션 —
</b>${ins.action}</p> <hr> <h3>키워드별 상세</h3> ${kwHtml} <p style="color:#999;font-size:.
85em;margin-top:2rem;">자동 생성 · 출처: 네이버 블로그/카페/뉴스</p> </div>`; } 강의 시연 포인트
keywords 시트에 자사 키워드 한 줄 추가 → dailyCollect() 즉시 실행 → 시트 채워지는 모습 weeklyReport() 즉시
실행 → 메일 도착 일부러 부정 키워드("환불"."후기 안 좋다") 추가 시 부정 즉시 Slack 알림 시연 → 위기관리 흐름 토
론 트러블슈팅 증상 원인 해결 네이버 API 401 Key 누락/오타 콘솔에서 새 Application 등록 확인 일 25,000 한도 초
과 키워드 너무 많음 키워드 5개 이내, 검색 sort=date 유지 AI가 모두 "중립" 본문 스니펫이 너무 짧음 가능하면 검색결
과 description 외 본문 일부 추가 (별도 fetch) 부정 알림 폭주 키워드가 혼한 단어 키워드를 정확한 브랜드/제품명으로
좁힘 응용 아이디어 자동 응답 초안: 네이버 블로그 부정 후기에 대해 CS 톤의 답변 초안 자동 작성 → 사람이 검토 후 발
송 경쟁사 비교 트렌드: 자사·경쟁사 멘션 추이를 차트로 시각화 제품팀 VOC 자동 라우팅: 주제가 "품질/UX"이면 제품팀,
"가격"이면 영업팀으로 자동 분기 인플루언서 스코어링: 자주 언급하는 블로거를 자동 식별해 협업 후보 리스트 작성

```

```
{responseMimeType:'application/json'}}}); return  
JSON.parse(JSON.parse(res.getContentText()).candidates[0].content.parts[0].text); } //  
----- // 2) 주간 리
```

포트 //

```
function weeklyReport() { const ss = SpreadsheetApp.openById(SHEET_ID); const all =  
ss.getSheetByName('mentions').getDataRange().getValues(); const head = all[0]; const since = new  
Date(Date.now() - 7*86400000); const week = all.slice(1).filter(r => new Date(r[0]) >= since); if  
(week.length === 0) return; const grouped = {}; let pos=0, neg=0, neu=0; const topicCount = {};  
week.forEach(r => { const [date, keyword, source, title, url, snippet, sentiment, topics] = r;  
grouped[keyword] = grouped[keyword] || {pos:0, neg:0, neu:0, items:[]};  
grouped[keyword].items.push({title, url, sentiment, topics, snippet}); if (sentiment === 'positive')  
{pos++; grouped[keyword].pos++;} else if (sentiment === 'negative') {neg++; grouped[keyword].neg+  
+;} else {neu++; grouped[keyword].neu++;} String(topics||'').split(',').filter(Boolean).forEach(t =>  
topicCount[t.trim()] = (topicCount[t.trim()]||0)+1); }); const topTopics =  
Object.entries(topicCount).sort((a,b)=>b[1]-a[1]).slice(0, 8); const insight = aiInsight({total:  
week.length, pos, neg, neu, topTopics, grouped}); const html = render(week.length, {pos,neg,neu},  
topTopics, grouped, insight); if (EMAIL_TO) GmailApp.sendEmail(EMAIL_TO, `[주간] 브랜드 멘션 다이제  
스트 — ${new Date().toISOString().slice(0,10)}`, '', {htmlBody: html});  
ss.getSheetByName('weekly_report').appendRow([new Date(), week.length, pos, neg, neu,  
JSON.stringify(insight)]); } function aiInsight(d) { const ctx = d.topTopics.map(([t,c])=>`${t}($  
{c})`).join(', '); const prompt = `브랜드 매니저용 주간 코멘트 작성. 한국어 250자 이내. 데이터: - 총 멘션: $  
{d.total} - 감성: 긍정 ${d.pos} / 부정 ${d.neg} / 중립 ${d.neu} - 상위 주제: ${ctx} JSON:  
{"summary":"...", "watch":"...", "action":"..."} - summary: 한 문장 종합 - watch: 다음 주 주의해서 지켜볼 한 가  
지 - action: 마케팅·CS·제품팀 중 한 곳에 보낼 권고 한 줄`; const url = `https://  
generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=$  
{GEMINI}`; const r = UrlFetchApp.fetch(url, {method:'post', contentType:'application/json', payload:  
JSON.stringify({contents:[{parts:[{text:prompt}]}], generationConfig:  
{responseMimeType:'application/json'}}}); return  
JSON.parse(JSON.parse(r.getContentText()).candidates[0].content.parts[0].text); } function  
render(total, sent, topTopics, grouped, ins) { const sumTotal = sent.pos + sent.neg + sent.neu || 1;  
const topicHtml = topTopics.map([t,c])=><li><b>${t}</b> · ${c}건</li>`).join(''); let kwHtml = '';  
for (const k in grouped) { const g = grouped[k]; const negs =  
g.items.filter(it=>it.sentiment==='negative').slice(0,3); const negHtml = negs.length ? <ul>${  
negs.map(n=><li><a href="${n.url}">${n.title}</a></li>`).join('')}</ul>` : <p style="color:#888;">  
부정 멘션 없음</p>`; kwHtml += <h4>${k}</h4> <p>긍정 ${g.pos} / 부정 ${g.neg} / 중립 ${g.neu}</p> <p  
style="color:#b45309;"><b>주의 멘션</b></p>${negHtml}`; } return <div style="font-family:'Noto  
Sans KR',sans-serif;line-height:1.7;max-width:740px;color:#222;"> <h2 style="border-bottom:2px  
solid #b45309;padding-bottom:.5rem;">브랜드 멘션 주간 다이제스트</h2> <p style="color:#6a604f;">${  
new Date().toLocaleDateString('ko-KR')} · 총 ${total}건</p> <h3>전체 감성</h3> <p>긍정 ${((sent.pos/  
sumTotal*100).toFixed(0))}% / 부정 ${((sent.neg/sumTotal*100).toFixed(0))}% / 중립 ${((sent.neu/  
sumTotal*100).toFixed(0))}%</p> <h3>상위 주제</h3> <ul>${topicHtml}</ul> <h3>🔴 핵심 인사이트</h3>  
<p><b>요약 — </b>${ins.summary}</p> <p><b>다음 주 관찰 — </b>${ins.watch}</p> <p><b>권고 액션 —  
</b>${ins.action}</p> <hr> <h3>키워드별 상세</h3> ${kwHtml} <p style="color:#999;font-size:.  
85em;margin-top:2rem;">자동 생성 · 출처: 네이버 블로그/카페/뉴스</p> </div>`; } 강의 시연 포인트  
keywords 시트에 자사 키워드 한 줄 추가 → dailyCollect() 즉시 실행 → 시트 채워지는 모습 weeklyReport() 즉시  
실행 → 메일 도착 일부러 부정 키워드("환불"."후기 안 좋다") 추가 시 부정 즉시 Slack 알림 시연 → 위기관리 흐름 토  
론 트러블슈팅 증상 원인 해결 네이버 API 401 Key 누락/오타 콘솔에서 새 Application 등록 확인 일 25,000 한도 초  
과 키워드 너무 많음 키워드 5개 이내, 검색 sort=date 유지 AI가 모두 "중립" 본문 스니펫이 너무 짧음 가능하면 검색결  
과 description 외 본문 일부 추가 (별도 fetch) 부정 알림 폭주 키워드가 혼한 단어 키워드를 정확한 브랜드/제품명으로  
좁힘 응용 아이디어 자동 응답 초안: 네이버 블로그 부정 후기에 대해 CS 톤의 답변 초안 자동 작성 → 사람이 검토 후 발  
송 경쟁사 비교 트렌드: 자사·경쟁사 멘션 추이를 차트로 시각화 제품팀 VOC 자동 라우팅: 주제가 "품질/UX"이면 제품팀,  
"가격"이면 영업팀으로 자동 분기 인플루언서 스코어링: 자주 언급하는 블로거를 자동 식별해 협업 후보 리스트 작성
```



```

// 2) 추가 키워드 판사 키워드 한 줄 추가 → dailyCollect() 즉시 실행 → 시트 채워지는 모습 weeklyReport() 즉시 실행
// → 메일 도착 일부터 부정 키워드("환불"."후기 안 좋다") 추가 시 부정 즉시 Slack 알림 시연 → 위기관리 흐름 토
론 도구 분석
function weeklyReport() {
  const ss = SpreadsheetApp.openById(SHEET_ID);
  const all = ss.getSheetByName(fetch.mentions).getDataRange().getValues();
  const head = all[0];
  const since = new Date(Date.now() - 7*86400000);
  const since = all.slice(1).filter(r => new Date(r[0]) >= since);
}

```


